

Efficient Approximation Algorithms for Adaptive Minimum Cost Seed Selection via mRR-set Updates

Chen Feng

The Hong Kong Polytechnic University
Hong Kong SAR, China
chenfeng@polyu.edu.hk

Gongyao Guo

The Hong Kong Polytechnic University
Hong Kong SAR, China
gong-yao.guo@connect.polyu.hk

Yiran Li

University of Toronto
Toronto, Canada
one.li@utoronto.ca

Jieming Shi*

The Hong Kong Polytechnic University
Hong Kong SAR, China
jieming.shi@polyu.edu.hk

Sibo Wang

The Chinese University of Hong Kong
Hong Kong SAR, China
swang@se.cuhk.edu.hk

Abstract

In a social network G with user costs $c(\cdot)$, the adaptive minimum cost seed selection (AMCS) problem aims to influence at least η users at minimum total cost, where seed users are selected iteratively based on observed diffusion. Prior work shows that truncating user influence by η is necessary for performance guarantees, and proposes multi-root reverse reachable sets (mRR-sets) to estimate truncated influence. However, to maintain estimation accuracy, all mRR-sets must be regenerated in each round to exclude influenced users, which limits scalability. Moreover, existing methods assume uniform user costs, inconsistent with practical scenarios.

In this paper, we attempt to solve AMCS under general user costs with high efficiency. To this end, we propose the EMASS framework which achieves an approximation ratio of $(\ln \eta + 1)^2 / [(1 - 1/e)^2 - \epsilon]$, where ϵ is the estimation error of mRR-sets. To generate mRR-sets efficiently, we develop a series of algorithms to update mRR-sets across rounds, which support the generation, root insertion, and cleaning of mRR-sets. In this way, we eliminate the need to regenerate mRR-sets in each round and thus the computation overhead is reduced significantly. Further, we prove that the updated mRR-sets are equal to the regenerated ones. Finally, extensive experiments on real social networks demonstrate that EMASS is faster than state-of-the-art methods by over an order of magnitude, while the influence target η is always reached with the total user cost among the lowest.

CCS Concepts

• **Theory of computation** → **Graph algorithms analysis**; • **Information systems** → **Data mining**.

ACM Reference Format:

Chen Feng, Gongyao Guo, Yiran Li, Jieming Shi, and Sibow Wang. 2026. Efficient Approximation Algorithms for Adaptive Minimum Cost Seed Selection via mRR-set Updates. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26)*, August 09–13,

*Jieming Shi is the corresponding author.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

KDD '26, Jeju Island, Republic of Korea

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2259-2/2026/08

<https://doi.org/10.1145/3770855.3818017>

2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 14 pages.
<https://doi.org/10.1145/3770855.3818017>

1 Introduction

Social networks have become a prevalent platform for users sharing information, opinions, products, etc. Exploiting the social relationships between users, advertisers can trigger a large influence cascade by recruiting only a small subset of users as *seeds*. When the number of influenced users reaches a certain threshold η , the influence can propagate spontaneously, achieving a viral effect and maximizing a campaign's impact [4, 14, 30, 32]. Clearly, the key to a successful campaign is reaching η . To this end, one may select users with large influence as the seeds. However, the cost of inviting an influential user is much higher than that of ordinary users, resulting in a large expense. For example, on YouTube, a nano-influencer with 10k to 50k followers charges \$20 to \$200 for creating a video, while a mega-influencer with over 1 million followers charges \$20,000 [22]. Thus, a more sensible strategy needs to be designed to reach η at a small cost.

In response to this demand, the minimum cost seed selection (MCSS) problem is studied to identify a set of seed users that reaches η at the minimum cost [9, 17, 20, 42]. A recent method CLEAR [9] for MCSS guarantees reaching η in expectation. That is, the average number of influenced users over many campaigns is η . Since the influence diffusion varies across campaigns, the influence of some campaigns will fall below η . The other method SCORE in [9] can reach η with a given probability p . However, we can never set $p = 1$ but $p = 1 - \epsilon_p$, since a shortfall ϵ_p should be deducted to accommodate the probability estimation error. Thus, SCORE actually cannot always reach η either. The failure of MCSS algorithms stems from their non-adaptive setting, where all seeds are selected upfront without observing the actual influence diffusion.

To address the limitations of MCSS, we study the *adaptive minimum cost seed selection* (AMCS) problem, where seeds are iteratively selected based on the influence diffusion from previous seeds. In this adaptive setting, seed selection continues until η users are observed to be influenced, ensuring that the influence threshold η is reached in every campaign. Note that the influence spread beyond η does not help address AMCS. Thus, Golovin et al. propose to truncate the influence at η [15, 16]. The state-of-the-art solution, ASTI [34], estimates the truncated influence with multi-root reverse reachable sets (mRR-sets), generated by performing breadth-first-search (BFS)

from random roots along the reverse direction of network edges. On this basis, ASTI selects seeds with the largest truncated influence in each round. It is proven that ASTI achieves an approximation ratio of $(\ln \eta + 1)^2 / [(1 - 1/e)^2(1 - \epsilon)]$, where ϵ is the estimation error of mRR-sets. Actually, ASTI only solves a special case of AMCS, since it assumes the user costs to be uniform, simplifying the objective of minimizing the total user cost to minimizing the number of seeds. Moreover, after each seed selection and diffusion, the influenced users do not need to be considered again and are thus removed, resulting in a residual graph. Then, some mRR-sets may be *polluted* by containing the deleted nodes. To ensure the estimation accuracy, ASTI regenerates all mRR-sets on the residual graph in each round, incurring significant computational overhead.

In this paper, we propose a new approach for AMCS, called EMASS (Efficient Minimum-Cost Adaptive Seed Selection). In each round, EMASS takes account of user costs by selecting b seeds that have high influence but low user cost. Rigorously, we require the selected seeds to approximate the optimal seed batch with a bounded gap. To this end, EMASS selects seeds in a “trial-and-error” manner. Starting from a few mRR-sets, we greedily select b seeds based on their costs and the number of mRR-sets including them. If the seeds are not qualified, we double the mRR-sets to try again. Otherwise, we diffuse from the seeds, and delete the influenced users for the next round of seed selection. This process goes on until η users are observed to be influenced. Thus, EMASS can always reach η . Besides, the total user cost of EMASS is proven to be at most $(\ln \eta + 1)^2 / [(1 - 1/e)^2(1 - \epsilon)]$ times that of the optimal algorithm, succeeding the approximation ratio of ASTI under non-uniform user costs.

At the core of EMASS is its update of mRR-sets for seed selection. In each round, rather than regenerating mRR-sets from scratch, we incrementally update the mRR-sets from the previous round. To this end, we should be able to trace the BFS process that generated an mRR-set from its roots, which forms a number of BFS trees. Accordingly, we modify the way to generate mRR-sets. First, we record not only the nodes traversed during BFS, but also the start position of each layer of the BFS trees. Second, we impose a sequence on the generation of trees to enhance the traceability. On this basis, we are ready to update the mRR-sets. Observe that unpolluted mRR-sets also have to be updated, since the number of roots needed in an mRR-set is increasing over rounds [34]. Thus, we insert roots to the unpolluted mRR-sets (e.g., R) and perform BFS from the new roots. Note that the new roots may coincide with nodes already in R . In this case, we mark the root as a virtual root, and do not perform BFS from it. Otherwise, the new BFS tree may entangle with existing trees in R . The virtual roots will be realized when R is polluted. For a polluted mRR-set, we traverse its trees according to their generation sequence, and preserve them before encountering a tree T^* that contains a deleted node in layer l^* . For the tree T^* , we preserve the layers above l^* , and start BFS from l^* to clean T^* . For the trees after T^* , we regenerate them from their roots. In this way, we complete the update of mRR-sets.

Our main contributions are summarized as follows.

- We study the AMCS problem under non-uniform user costs and propose the EMASS algorithm for solution, which achieves an approximation ratio of $(\ln \eta + 1)^2 / [(1 - 1/e)^2(1 - \epsilon)]$.
- Instead of regenerating mRR-sets in each round, we obtain mRR-sets by updating previous ones. To this end, we modify the mRR-

set generation process by recording the structure of BFS trees and imposing a fixed sequence on the BFS process.

- To update unpolluted mRR-sets, we insert new roots into them, and introduce the concept of virtual roots to resolve possible conflict. For polluted mRR-sets, we preserve the unpolluted BFS trees and regenerate the polluted part.
- Extensive experiments demonstrate that EMASS is faster than existing methods that reach η in every campaign, often by over an order of magnitude, while the total cost is among the lowest.

2 Preliminaries

2.1 Influence Diffusion and Realization

The social network is modeled as a directed graph $G = (V, E)$, where V is the set of users and E is the set of edges with $|V| = n$ and $|E| = m$. The cost¹ to select a user $u \in V$ as a seed is $c(u) \in (0, 1]$. Each edge $e = (u, v) \in E$ is associated with a diffusion probability $p(e) \in [0, 1]$. For any edge $e = (u, v)$, we say u is the in-neighbor of v , and v is the out-neighbor of u . The set of in-neighbors (resp. out-neighbors) of u is denoted by $N_{\text{in}}(u)$ (resp. $N_{\text{out}}(u)$).

The influence diffusion starts from a set of seeds $S \subseteq V$, and is depicted by the *Independent Cascade (IC) model* [24]. Initially, at time step 0, only the users in S are activated, while other users are inactive. At time step $t > 0$, for each user u activated in the last step $t - 1$, u has a single chance to activate its out-neighbor $v \in N_{\text{out}}(u)$ via edge (u, v) , which succeeds with probability $p(u, v)$. If successful, v becomes activated. Otherwise, v remains inactive. Note that all activation attempts are independent, and once a user is activated, it remains active. The diffusion process proceeds until no user is further activated. The number of activated users is the influence spread of S , denoted by $I(S)$.

Given the probabilistic nature of edge activations, $I(S)$ is inherently a random variable. To ascertain each edge (u, v) , we can flip a coin of bias $p(u, v)$. If it shows the head, the edge (u, v) is said to be “live” (encoded as “1”), and v can be activated by u when u is active. Otherwise, the edge is “blocked” (encoded as “0”), and v will not be activated by u . Before the toss of coins, the edge states are unknown, denoted as a random variable Φ . The ascertained states of edges are called a realization $\phi : E \rightarrow \{0, 1\}$. The probability that Φ is revealed to coincide with some realization ϕ is p_ϕ . Under a realization ϕ , the influence spread of seed set S is deterministic, denoted as $I(S, \phi)$.

2.2 Problem Formulation

The Adaptive Setting. We solve the AMCS problem adaptively, where seed selection alternates with influence diffusion in an iterative manner. In each round i , we select a batch of b seeds S_i based on the diffusion results from previous rounds. Then, the seeds in S_i start to influence their neighbors, and the diffusion of round i starts. Let the set of activated users be A_i . According to the IC model, users in A_i will not be activated again, and have applied their only chance to activate their neighbors. Then, A_i cannot expand the influence spread in subsequent rounds. Thus, we remove A_i and their edges from the current graph, resulting in the residual graph

¹Without loss of generality, we assume costs are in $(0, 1]$ since any non-negative cost can be rescaled to $[0, 1]$ and users with zero cost can always be selected first without impairing the performance of algorithms.

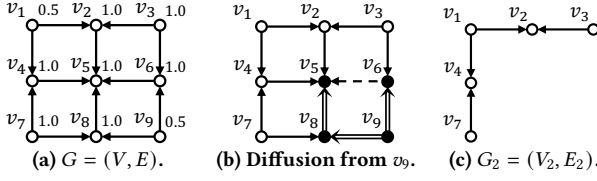


Figure 1: An example of a partial realization and AMCS.

$G_{i+1} = (V_{i+1}, E_{i+1})$ for the next round of seed selection. The seed set S_i is also incorporated into the seeds of previous $i - 1$ rounds, forming S_i . The above process continues until η is reached. In the special case of $i = 0$, we have $S_0 = \emptyset, S_0 = \emptyset, A_0 = \emptyset$, and the residual graph $G_1 = G$. Further, we denote $n_i = |V_i|$ and $n_1 = n$.

The AMCS Problem. Due to the randomness in the influence diffusion of previous rounds, the seeds to be selected should adapt to the observed diffusion results. Accordingly, the solution for an adaptive problem is called a *policy*, which returns seed users based on the previous diffusion results. When a policy finishes, a realization is revealed by the diffusion from the selected seeds. Given a policy π and realization ϕ , the seeds selected by π are denoted as $S(\pi, \phi)$. Then, the expected cost of policy π can be written as $c(\pi) = \sum_{\phi} p_{\phi} \sum_{s \in S(\pi, \phi)} c(s)$. On this basis, we define the AMCS problem in Definition 2.1. Note that AMCS requires policies to reach η under any realization. Should a policy fails in any realization, it cannot be considered feasible. The NP-hardness of AMCS is immediate, since it generalizes the non-adaptive case, which is already NP-hard [28].

Definition 2.1 (AMCS). Given the graph $G = (V, E)$, user costs $c(\cdot)$, diffusion probabilities $p(\cdot)$, batch size b , and an influence threshold η , AMCS aims to find a policy π^o that reaches η in every possible realization with the minimum expected cost: $\pi^o = \arg \min_{\pi} c(\pi)$ s.t. $I(S(\pi, \phi), \phi) \geq \eta, \forall \phi$.

Truncated Influence. In AMCS, we are only concerned with reaching η in each realization. Any influence spread beyond η is unnecessary [34]. Thus, given a seed set S , we only consider its truncated influence for any realization ϕ . In each round i , we have selected seeds S_{i-1} in previous $i - 1$ rounds, activating $I(S_{i-1}, \phi)$ users and leaving only $\eta_i = \eta - I(S_{i-1}, \phi)$ users to be activated. Meanwhile, the number of users that can be activated by S is $I(S \cup S_{i-1}, \phi) - I(S_{i-1}, \phi)$. Note that any influence exceeding η_i implies the overall influence will surpass η . Therefore, we truncate the influence of S by η_i , defining $\Gamma_i(S, \phi) = \min\{I(S \cup S_{i-1}, \phi) - I(S_{i-1}, \phi), \eta_i\}$. Accordingly, the expected truncated influence of S in round i is $\Gamma_i(S) = \mathbb{E}_{\Phi}[\Gamma_i(S, \Phi)]$. Especially, for the first round, we have $\Gamma_1 = \Gamma$ and $\eta_1 = \eta$.

mRR-sets. To select influential users as seeds, we need to estimate the truncated influence of users. To this end, we adopt the mRR-set method proposed in [34]. Given a residual graph G_i , the generation of an mRR-set takes 3 steps: (1) select $k_i = n_i / \eta_i$ users uniformly at random from V_i as roots; (2) simulate stochastic BFS from these k_i roots along reversed edge directions; (3) include all visited users and the roots as the mRR-set. For any set of users S , we say an mRR-set R is covered by S if $S \cap R \neq \emptyset$. The coverage of S , denoted as $\Lambda(S)$, is the number of mRR-sets covered by S . Given θ mRR-sets, $\Gamma_i(S)$

²When $\eta_i = 1$, there is no need to generate mRR-sets for seed selection. We simply select the user with the smallest cost as the seed.

Algorithm 1: EMASS

Input: Graph G , costs $c(\cdot)$, diffusion probabilities $p(\cdot)$, threshold η , batch size b , estimation error ϵ , failure probability δ .
Output: A seed set S .

```

1  $i \leftarrow 1, G_i \leftarrow G, n_i \leftarrow n, \eta_i \leftarrow \eta, S \leftarrow \emptyset;$ 
2 while  $\eta_i > 0$  do
3    $S_i \leftarrow \emptyset, \mathcal{R}_i \leftarrow \emptyset, \hat{\epsilon} \leftarrow \frac{\epsilon - \delta}{1 - \delta}, \theta_{\max} \leftarrow 2(1 + \frac{\epsilon}{3})n_i \ln \frac{6}{\delta} / \hat{\epsilon}^2;$ 
4    $W \leftarrow \lceil \log_2 \frac{n_i}{\hat{\epsilon}^2} \rceil, \theta \leftarrow \lceil \frac{\theta_{\max} \hat{\epsilon}^2}{n_i} \rceil, a_1 \leftarrow \ln \frac{3Wn_i}{\delta}, a_2 \leftarrow \ln \frac{3W}{\delta};$ 
5   while True do // select  $b$  seeds
6     if  $|\mathcal{R}_{i-1}| > \theta$  then // update mRR-sets
7       Update  $\theta$  mRR-sets of  $\mathcal{R}_{i-1}$  as  $\mathcal{R}_i;$ 
8     else
9       Update  $\mathcal{R}_{i-1}$  and generate  $\theta - |\mathcal{R}_{i-1}|$  mRR-sets as  $\mathcal{R}_i;$ 
10    for  $j \leftarrow 1, 2, \dots, b$  do // greedy selection
11       $u \leftarrow \arg \max_{v \in V_i \setminus S_i} [\Lambda(S_i \cup \{v\}) - \Lambda(S_i)] / c(v);$ 
12       $S_i \leftarrow S_i \cup \{u\};$ 
13       $\Lambda^l(S_i) = \left( \sqrt{\Lambda(S_i) + \frac{2a_1}{9}} - \sqrt{\frac{a_1}{2}} \right)^2 - \frac{a_1}{18};$  // lower bound
14       $\Lambda^u(S_i) = \left( \sqrt{\frac{\Lambda(S_i)}{1-1/e} + \frac{a_2}{2}} + \sqrt{\frac{a_2}{2}} \right)^2;$  // upper bound
15      if  $\theta < \theta_{\max}$  and  $\Lambda^l(S_i) < (1 - \hat{\epsilon})(1 - 1/e)\Lambda^u(S_i)$  then
16         $\theta \leftarrow 2\theta;$  // double the mRR-sets needed
17      else
18        break;
19     $S \leftarrow S \cup S_i;$ 
20    Diffuse from  $S_i$ , delete activated nodes and their edges from  $G_i;$ 
21    Take the residual graph as  $G_{i+1};$ 
22     $\eta_{i+1} \leftarrow \eta_i - (n_i - n_{i+1}); i \leftarrow i + 1;$ 
23 return  $S;$ 
```

can be estimated as $\tilde{\Gamma}_i(S) = \eta_i \Lambda(S) / \theta$. The estimation accuracy of $\tilde{\Gamma}_i(S)$ is proven to be $(1 - 1/e) \Gamma_i(S) \leq \mathbb{E}[\tilde{\Gamma}_i(S)] \leq \Gamma_i(S)$ [34].

Example 2.2. Fig. 1 illustrates AMCS with a graph $G = (V, E)$, where $\eta = 5, b = 1, p(e) = 0.5 (\forall e \in E)$, and all user costs are 1.0 except $c(v_1) = c(v_9) = 0.5$. For solution, we select users with small cost but large influence as seeds. In round 1, by simple calculation, we find $v_{1,3,7,9}$ have the same influence and do not need to be truncated, while $c(v_9)$ is smaller. Thus, we select v_9 as the seed. Then, a diffusion starts from v_9 and probes edges $(v_9, v_6), (v_9, v_8), (v_6, v_5)$, and (v_8, v_5) in Fig. 1b. As a result, (v_6, v_5) (dashed arrows) is revealed to be blocked while the other three edges (double lines) are live, activating 4 users $v_{9,6,8,5}$ and reducing η to 1. To seed in round 2, we delete the activated users and their edges, deriving the residual graph $G_2 = (V_2, E_2)$ in Fig. 1c. After truncating with $\eta_2 = 1$, all users have the same truncated influence while v_1 has the least cost. Thus, we select v_1 as the seed to start diffusion. Whatever the diffusion result is, we can reach η and finish the seeding process, since v_1 influences at least itself.

3 The EMASS Framework

We propose the EMASS policy to solve AMCS, which considers non-uniform user costs and obtains mRR-sets by updating previous ones. In each round i , we select seeds in a “trial-and-error” manner. For each trial, we greedily select b seeds (forming S_i) from V_i , based on their coverage-to-cost ratios. Then, we examine whether S_i approximates the optimal seed set S^o with a bounded error. If S_i

is qualified, we take S_i as the seeds. Otherwise, the mRR-sets are doubled for a new trial, until a qualified seed set is selected.

The pseudo code of EMASS is in Alg. 1. To begin with, we initialize G_i as G , η_i as η , and the overall seed set S as empty. In each round i , if the remaining influence threshold to be reached η_i is positive, we will continue seed selection. For preparation, we derive the maximum number of mRR-sets $\theta_{\max}$ needed to select a qualified seed batch S_i , the initial number of mRR-sets θ , and the parameters a_1 and a_2 used in bound calculation (Lines 3-4).

In Lines 3-19, we select a qualified seed set S_i by trials, which should approximate the optimal seed set S^o with bounded error. In each attempt, we first derive θ mRR-sets by updating θ previous ones in $\mathcal{R}_{i-1}$ (Line 7). If $|\mathcal{R}_{i-1}| < \theta$, we not only update the mRR-sets in $\mathcal{R}_{i-1}$, but also generate new mRR-sets as a supplement (Line 9). The detailed process to update an mRR-set will be introduced in Section 4. Based on the mRR-sets, we select b seeds into S_i sequentially (Lines 10-12). Specifically, for the j -th selection where $j - 1$ seeds are already in S_i , the node with the largest marginal coverage-to-cost ratio is selected. Next, we calculate the lower bound of selected seeds (i.e., $\Lambda^l(S_i)$) and the upper bound of S^o (i.e., $\Lambda^u(S^o)$) in Lines 13 and 14 respectively. If the lower bound $\Lambda^l(S_i)$ is far below the upper bound $\Lambda^u(S^o)$ while $\theta < \theta_{\max}$ (Line 15), the selected seeds are not qualified, and we double the mRR-sets for the next trial (Line 16). Otherwise, the seed set S_i is qualified. Thus, we stop the trial for this round and insert S_i to the cumulative seed set S (Line 19). In Lines 20-22, by the adaptive setting in Sec. 2.2, we trigger influence diffusion from S_i and delete the activated nodes as well as their edges, resulting in a new residual graph G_{i+1} . The influence threshold to be achieved is also decreased to be η_{i+1} .

The above process will not stop until the remaining influence threshold is non-positive. The overall seed set S output by EMASS is a result specific to the realization revealed when running EMASS.

3.1 Theoretical Analysis

To evaluate the performance of EMASS theoretically, we present the approximation ratio of EMASS in Theorem 3.1, whose proof is in the technical report [10].

THEOREM 3.1. *For any policy π that spends the same user cost as EMASS in each round, the total user cost of EMASS to reach η approximates that of π by $c(\text{EMASS}) \leq \frac{(\ln \eta + 1)^2}{(1 - 1/e)^2(1 - \epsilon)} c(\pi)$.*

For time complexity, we first measure the complexity of ASTI solving AMCS in Lemma 3.2, where $c_{\max} = \max_{v \in V} c(v)$, $c_{\min} = \min_{v \in V} c(v)$ and the proof is in [10]. Let γ be the number of nodes activated in round $i - 1$. We provide the time complexity of EMASS in Theorem 3.3, whose proof is in Appendix A.4.

LEMMA 3.2. *The expected running time of ASTI solving AMCS in the i -th round is $O\left(\frac{c_{\max}}{c_{\min}} \cdot \frac{(m_i + n_i) \ln n_i}{\epsilon^2}\right)$.*

THEOREM 3.3. *Given the number of nodes γ activated in round $i - 1$ ($i > 1$), in the i -th round, the expected time complexity of EMASS is*

$$O\left(\frac{\gamma}{\eta_{i-1} - 1} \cdot \frac{c_{\max}}{c_{\min}} \cdot \frac{(m_i + n_i) \ln n_i}{\epsilon^2}\right).$$

Note that $\gamma \leq \eta_{i-1} - 1$ (i.e., $\frac{\gamma}{\eta_{i-1} - 1} \leq 1$), otherwise round i will not be carried out. The efficiency of EMASS arises from the mRR-set

revision techniques in EMASS in Section 4, which reduce the time by a factor of $\frac{\gamma}{\eta_{i-1} - 1}$. In practice, the number of nodes activated in a single round (i.e., γ) is typically small compared to the remaining η_{i-1} , indicating significant efficiency improvement.

4 mRR-set Updates

In each round i , EMASS requires a collection of mRR-sets $\mathcal{R}_i$ for seed selection. Instead of generating new mRR-sets from scratch, we derive $\mathcal{R}_i$ by updating previous ones. Specifically, the update mechanism includes the following three components:

- **mRR-sets generation.** In the first round, no mRR-sets exist, and in a later round i the previous mRR-sets may be not enough. In both cases, fresh new mRR-sets need to be generated.
- **Root insertion.** The number of roots $k_i = \frac{n - I(S_{i-1}, \phi)}{\eta - I(S_{i-1}, \phi)}$ required in each mRR-set is increasing with round i , since $I(S_{i-1}, \phi)$ increases when more seeds are selected over rounds.
- **mRR-set cleaning.** In the last round $i - 1$, the mRR-sets are obtained before deleting newly activated nodes to derive G_i . Thus, some mRR-sets may contain the deleted nodes (called polluted), and need to be cleaned.

The correctness of our update mechanism is guaranteed by the lemmas in this section, whose proofs are in the Appendix.

4.1 mRR-sets Generation with Hierarchy

The Hierarchical Data Structure. When generating an mRR-set R , the reverse BFS process from each root results in a tree T , where the adjacent two layers are parents and their children. To keep track of this relationship, we store the mRR-sets in a hierarchical structure, rather than the unordered set in [34]. The trees from different roots will not overlap with each other, since each node has only one parent. Thus, the trees from different roots constitute a forest F . For efficient memory usage, we do not store each layer of a tree as an array, but store the nodes of T in a single array, according to their sequence of being visited. Meanwhile, we apply a new array L to store the position of each layer's first node in T . Combining T and L , we can restore the hierarchical structure of the BFS tree. For the whole forest F with k trees, we pile up the user arrays of each tree to store the nodes in the forest, resulting in a two-dimensional array $F = \{T_1, T_2, \dots, T_k\}$. Similarly, the layer information of each tree is piled up to record the hierarchical information of F , denoted as $H = \{L_1, L_2, \dots, L_k\}$. Then, the mRR-set corresponding to this forest can be written as $R = \{F, H\}$, and a collection of θ mRR-sets is $\mathcal{R} = \{R_1, R_2, \dots, R_\theta\}$. To accommodate the previous terminology, we say a set of users S covers $R = \{F, H\}$ if $F \cap S \neq \emptyset$.

Fixed Generation Sequence. To facilitate efficient mRR-sets revision, we impose a sequence on the reverse BFS process that generates an mRR-set. The sequence is applied to not only the tree level but also the node level. Specifically, each time we only take out one root and generate one BFS tree. That is, the trees are sequentially generated, and the newly generated trees will not affect but build on the older ones. For each node, the probing of its in-neighbors follows some fixed order, e.g., the ascending order of node IDs.

The mRR-set Generation Algorithm. Based on the above idea, we devise Alg. 2 to generate an mRR-set. To begin with, we calculate the required root number k_i , and the probability q to round up k_i (Line 1). Then, we declare two empty two-dimensional arrays F

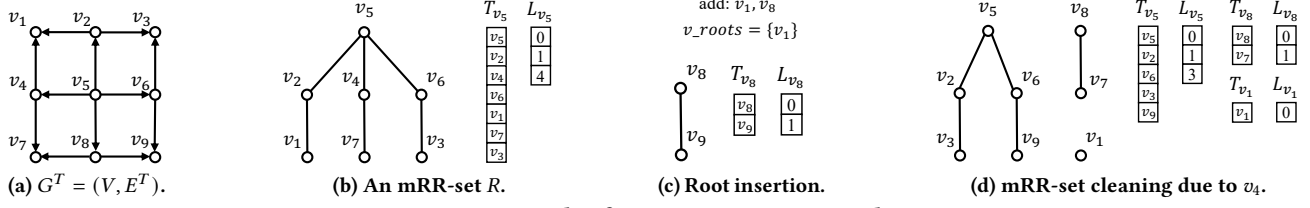


Figure 2: An example of mRR-set generation and revision.

Algorithm 2: Generate(G_i, η_i)**Input:** Graph $G_i = (V_i, E_i)$, diffusion prob. $p(\cdot)$, threshold η_i .**Output:** An mRR-set R with users F and hierarchy H .

```

1  $k_i \leftarrow |V_i|/\eta_i, q \leftarrow k_i - \lfloor k_i \rfloor$ ;
2 arrays  $F \leftarrow \emptyset, H \leftarrow \emptyset$ ;  $visited[v] \leftarrow false, \forall v \in V_i$ ;
3 if  $rand() < q$  then
4    $k_i \leftarrow \lfloor k_i \rfloor + 1$ ;
5  $roots \leftarrow$  select  $k_i$  users from  $V_i$  uniformly at random;
6 for  $u \in roots$  do
7   arrays  $T \leftarrow \{u\}, L \leftarrow \{0\}, visited[u] \leftarrow true$ ;
8   Explore( $T, L, V_i, visited, 0, 1$ );
9    $F.append(T), H.append(L)$ ;
10 return  $R = \{F, H\}$ ;
```

Procedure Explore($T, L, V, visited, head, beg$)

```

1  $h \leftarrow head$ ;
2 while  $i < T.size()$  do
3    $v \leftarrow T[h]$ ;
4   for  $w \in N_{in}(v)$  do
5     if  $visited[w] = false$  and  $w \in V$  then
6       if  $rand() < p(w, v)$  then
7          $T.append(w)$ ;
8          $visited[w] \leftarrow true$ ;
9    $h \leftarrow h + 1$ ;
10 if  $h == beg$  then
11    $L.append(beg)$ ;
12    $beg \leftarrow T.size()$ ;
```

and H to store the nodes and the hierarchical information, and a boolean array $visited$ which indicates whether a node is already in the mRR-set (Line 2). Next, we determine k_i and select k_i random users from V_i as the roots. For each root u , we initialize the user array T to contain only u itself. Note that the root itself forms the first layer, the beginning position of which is 0 in T , leading to $L = \{0\}$. After marking u as visited, we proceed to expand the BFS by calling the Explore procedure.

Specifically, we set the start position of BFS in T to be $head = 0$, and the start position of the next layer $beg = 1$. Then, at each time h , we take out the h -th user v from T , trying to visit its in-neighbors (e.g. w) with probability $p(w, v)$ that have not been visited yet (Lines 3-5). If the visit is successful, we append w to T for further probing and mark it as visited (Lines 6-8). When the next position $h + 1$ to probe is exactly beg , we reach the end of the current layer, also the begin of the next layer. Thus, we append beg to L , and update beg as $T.size()$ (Lines 10-12). The above process goes on until all nodes in T have been visited (i.e., $i == T.size()$).

Note that the sequence imposed on mRR-set generation does not impair the estimation accuracy, since the accuracy performance of

mRR-sets only relies on the existence of nodes in an mRR-set R , which is proven to be unaffected in Lemma 4.1.

LEMMA 4.1. *Let $R = \{F, H\}$ and $R' = \{F', H'\}$ be the mRR-sets generated from the same set of roots with different probing sequences. Then, $\forall U \subseteq V_i$, we have $\mathbb{P}[F = U] = \mathbb{P}[F' = U]$.*

Example 4.2. Given the transposed graph G^T in Fig. 2a, suppose node v_5 is randomly selected as the root. We illustrate how to generate a BFS tree by Alg. 2. First, we initialize the user array $T_{v_5} = \{v_5\}$, the layer information $L_{v_5} = \{0\}$, and the next layer's start position $beg = 1$. A reverse BFS is performed from v_5 over G^T . The first node v_5 is taken out from T_{v_5} , and its in-neighbors $v_{2,4,6,8}$ are probed. Assume only the visit to v_8 fails. Then, the other three nodes are appended to T_{v_5} and marked as visited. After that, the probing of the first layer $\{v_5\}$ is completed, and the next layer $\{v_2, v_4, v_6\}$ is generated. Accordingly, beg is appended to L_{v_5} and updated as $beg = T_{v_5}.size() = 4$. For the next layer, suppose $v_{1,7,3}$ are visited from $v_{2,4,6}$ respectively, and appended to T_{v_5} . This completes the probing of the second layer, so $beg = 4$ is appended to L_{v_5} . The process then terminates as no more nodes can be probed.

4.2 Root Insertion

Virtual Roots. Let us consider an arbitrary mRR-set R from the previous round $i - 1$, whose roots are denoted as K with $|K| = k_{i-1}$. Recall that when generating a new mRR-set, the roots are selected before any BFS process. To mimic this feature, we assume new roots were inserted when the k_{i-1} roots of R had just been selected. Then, the selection of new roots only needs to be different from K . As a result, the nodes in $R \setminus K$ may be selected as a new root, invalidating existing tree structure of R . If we attempt to correct R right now, significant revisions are required, incurring much computation time. To circumvent this problem, we propose to record the root that coincides with an existing user in R as a virtual root, and postpone revising R until R needs to be revised due to pollution. Note that the virtual roots will be realized only when R is polluted. That is the virtual roots are still counted as roots of R , since they are just waiting to be realized. Lemma 4.3 shows that our inserted roots are equivalent to those directly selected from V_i .

LEMMA 4.3. *Given k_{i-1} random roots from V_{i-1} while valid in V_i , we further select $k_i - k_{i-1}$ different roots from V_i . Then, the k_i roots have the same distribution as the k_i roots directly selected from V_i .*

The Root Insertion Algorithm. Based on the above idea, we introduce how to insert k new roots into an mRR-set $R = \{F, H\}$ in Alg. 3. To begin with, we traverse the forest F , and mark each node in R as $visited$ (Lines 1-3). Then, we select k random nodes from V_i as the new roots, which should not coincide with existing roots. Next, we attempt to generate BFS trees from the new roots. Specifically,

Algorithm 3: Insert(R, k)

Input: Graph $G_i = (V_i, E_i)$, diffusion probability $p(\cdot)$, mRR-set $R = \{F, H\}$, virtual roots v_roots , number of new roots k .
Output: A new mRR-set R .

```

1  $visited[v] \leftarrow false, \forall v \in V_i$ ;
2 for  $v \in F$  do
3    $visited[v] \leftarrow true$ ;
4  $new\_roots \leftarrow k$  non-root random nodes from  $V_i$ ;
5 for  $iter \leftarrow 0 \cdots k-1$  do
6    $r \leftarrow new\_roots[iter]$ ;
7   if  $visited[r] == true$  then
8      $v\_roots.append(r)$ ;
9   else
10    arrays  $T \leftarrow \{r\}, L \leftarrow \{0\}, visited[r] \leftarrow true$ ;
11    Explore( $T, L, V_i, visited, 0, 1$ );
12     $F.append(T), H.append(L)$ ;
13 return  $R$ ;
```

each time, we take out a new root r (Line 6). If r is already in R (i.e., $visited[r] == true$), then it is a virtual root and thus we store r into the array of virtual roots v_roots (Line 8), waiting for future materialization. Conversely, if r is not in R , we initialize a tree array T and an information array L for it (Line 10). To build a tree from r , we invoke the Explore procedure. Accordingly, a new tree T along with its layer information L is generated and inserted into F and H . After all new roots are traversed, we return the new mRR-set R . The correctness of our root insertion method is validated by Lemma 4.4.

LEMMA 4.4. *Given a random mRR-set R' with k_{i-1} roots from G_{i-1} , we insert $k_i - k_{i-1}$ roots to R' by Alg. 3. Then, the distribution of R' and that of a fresh mRR-set R generated from G_i are the same.*

Example 4.5. To illustrate Alg. 3, we insert $k = 2$ new roots to the mRR-set R in Fig. 2b. Let v_1 and v_8 be the new roots selected from the graph. To generate new trees, we start from v_1 but find that it is already in R . Thus, we insert it to the virtual roots of R , resulting in $v_roots = \{v_1\}$. For the next root v_8 , it is not in R . Therefore, we trigger a BFS process from v_8 by calling the Explore procedure. As a result, only v_9 is visited, and is the only node in the second layer. Accordingly, the user array of the new tree is $T_{v_8} = \{v_8, v_9\}$, and the layer information is $L_{v_8} = \{0, 1\}$, are shown in Fig. 2c.

4.3 mRR-set Cleaning

Main Idea. Consider an mRR-set R from round $i - 1$ polluted by containing nodes that are non-existent in the current graph G_i . To handle R , a simple way is to delete it and generate a new one for replacement. However, Lemma 4.6 shows that this method incurs evident estimation error, since the distribution of remaining mRR-sets becomes different from that of new mRR-sets. Actually, the correctness of mRR-sets relies on two parts: the uniform distribution of roots, and the stochastic reverse BFS process. To ensure a correct root distribution, Lemma 4.7 shows that we can delete the polluted roots from R , and supplement R with new roots from V_i .

LEMMA 4.6. *Given a random mRR-set R' from G_{i-1} , and a random new mRR-set R from G_i , we have $\mathbb{P}[u \in R'] \neq \mathbb{P}[u \in R], \forall u \in V_i$.*

LEMMA 4.7. *Given a set of k roots K randomly selected from V_{i-1} , we delete the last k' roots from K , yielding K' . Then, K' has the same distribution as $k - k'$ roots randomly selected from V_i .*

Note that, the non-existent nodes often only pollute part of the trees in $R = \{F, H\}$. For efficiency, we attempt to reuse the unpolluted trees to avoid regenerating them. Let T^* be the first polluted tree that contains a non-existent node x in layer l^* . Recall that the trees in R were generated sequentially, each based on previous trees. Then, the trees generated after T^* are dependent on and are affected by T^* . Thus, we regenerate the unpolluted trees after T^* . The trees before T^* are not affected, and we simply preserve them. For T^* itself, we retain the layers above l^* , just like retaining the trees before T^* . Further, we argue that the layer l^* can also be preserved after erasing x . To explain, let us recall the BFS process that generated T^* . Before x began to visit its in-neighbors, the edges through which the nodes in $l^* \setminus \{x\}$ were visited, had already been probed to be live. Thus, the nodes in $l^* \setminus \{x\}$ are actually not affected by x . Similarly, the edges probed to be blocked when producing l^* were also not affected, and thus no new user will be added to l^* . Differently, the layers after l^* in T^* need to be regenerated by starting BFS from $l^* \setminus \{x\}$, and the trees after T^* need to be regenerated from their roots. If a root itself is polluted, we replace it by with a new one. And the virtual roots after l^* will be realized.

The mRR-set Cleaning Algorithm. With the above idea, we propose our cleaning method in Alg. 4. Given the residual graph G_i and a polluted mRR-set $R = \{F, H\}$, we first traverse the user arrays F of R to find the first deleted node x which is not in V_i (Lines 2-9). Before we reach the first deleted node x , the trees can be preserved and thus the nodes therein are marked as visited (Line 4). When we find x , we record the index t of the polluted tree T^* (i.e., $F[t] = T^*$), and the layer information of T^* is $H[t]$. Meanwhile, we record the index of the polluted node x in T^* as idx (i.e., $T^*[idx] = x$). To preserve l^* , we locate the beginning position of l^* in $F[t]$ (denoted as beg), by performing a binary search of x 's index in $H[t]$, since $H[t]$ records the beginning position of layers of T^* and is thus strictly increasing. Then, the beginning position of the next layer in $F[t]$ is $H[t][beg + 1]$. Next, in Lines 9-11, we collect the roots of trees after T^* as $roots$ for further regeneration.

By the above lines, we obtain necessary information for subsequent mRR-set preservation and regeneration. To preserve the trees before T^* , we truncate the node array F and layer information H to T^* 's index t (Line 12). For the tree T^* , we truncate its node array $F[t]$ to the end of l^* , i.e., $H[t][beg + 1]$ (Line 13). The layer information is accordingly truncated to the begin of l^* , i.e., beg (Line 14). After deleting non-existent nodes from l^* (Line 15), we insert the virtual roots of R that are after T^* (i.e., the unvisited ones) to $roots$ for subsequent materialization. Next, we regenerate layers below l^* in T^* by applying the Explore procedure (Line 22). Finally, for each root in $roots$, we generate a new tree in Lines 23-26 like Alg. 2. The correctness of the cleaning method is in Lemma 4.8.

LEMMA 4.8. *Given a polluted mRR-set from G_{i-1} , we clean it by Alg. 4 and obtain the revised mRR-set R' . Then, the distribution of R' is the same as an mRR-set R generated from G_i with the same roots.*

Example 4.9. Let us consider the mRR-set $R = \{F, H\}$ in Fig. 2b combined with the trees inserted in Fig. 2c, where $F = \{T_{v_5}, T_{v_8}\}$

Algorithm 4: Clean(R)

Input: Graph $G_i = (V_i, E_i)$, diffusion probabilities $p(\cdot)$, the mRR-set $R = \{F, H\}$, virtual roots v_roots .
Output: A clean mRR-set R .

```

1  $t, idx, beg \leftarrow 0$ ;  $roots \leftarrow \emptyset$ ;  $visited[v] \leftarrow false, \forall v \in V_i$ ;
2 for  $t \leftarrow 0, 1, 2, \dots, F.size() - 1$  do
3   for  $x \in F[t]$  do
4      $visited[x] = true$ ;
5     if  $x \notin V_i$  then
6        $idx \leftarrow \text{index of } x \text{ in } F[t]$ ;
7        $beg \leftarrow \text{BinarySearch}(H[t], idx)$ ;
8       goto Line 9;           // Break nested loop
9 for  $j \leftarrow t, t+1, \dots, F.size() - 1$  do
10  if  $F[j][0] \in V_i$  then
11     $roots.append(F[j][0])$ ;
12  $F \leftarrow F[0, \dots, t], H \leftarrow H[0, \dots, t]$ ; // Keep first  $t+1$  trees
13  $F[t] \leftarrow F[t][0, \dots, H[t][beg+1]]$ ; // Truncate  $F[t]$ 
14  $H[t] \leftarrow H[t][0, \dots, beg]$ ; // Truncate  $H[t]$ 
15 Erase all deleted users from  $F[t]$ ;
16 for  $v \in v\_roots$  do
17  if  $v \notin V_i$  or  $visited[v] == false$  then
18    delete  $v$  from  $v\_roots$ ;
19  if  $visited[v] == false$  then
20     $roots.append(v)$ ;
21  $visited[v] \leftarrow true, \forall v \in v\_roots \cup roots \cup F[t]$ ;
22 Explore( $F[t], H[t], V_i, visited, beg, F[t].size()$ );
23 for  $r \in roots$  do
24  arrays  $T \leftarrow \{r\}, L \leftarrow \{0\}, visited[r] \leftarrow true$ ;
25  Explore( $T, L, V_i, visited, 0, 1$ );
26   $F.append(T), H.append(L)$ ;
27 return  $R$ ;
```

and $H = \{L_{v_5}, L_{v_8}\}$. We assume node v_4 was activated and deleted in the previous round. Then, we clean R by Alg. 4. To begin with, we traverse the user arrays in F to find the deleted user. When we reach $F[0][2] = T_{v_5}[2]$, we find v_4 is not in V_i and thus record its index in T_{v_5} by $idx = 2$ (Line 6). Then, the layer that contains v_4 is $l^* = \{v_2, v_4, v_6\}$. To derive the begin position beg of l^* in T_{v_5} , we conduct binary search of v_4 's index (i.e., $idx = 2$) in the layer array $H[0]$, resulting in $beg = 1$ (Line 7). Then, we obtain all the reusable nodes and begin to collect the roots after $F[0]$ by $roots = \{v_8\}$ (Line 11), from which new BFS will be triggered. After that, we truncate R till $F[0]$ and $H[0]$ (Line 12). For $T^* = F[0]$ that exactly contains v_4 , we truncate it to the layer l^* and erase v_4 from $F[0]$ (Lines 13-15). Note that R contains a virtual root v_1 , which is after v_4 in $F[0]$. Thus, we include v_1 into $roots = \{v_8, v_1\}$ (Line 20). For regeneration, we first complete the BFS in $F[0]$ by applying Explore to start from the beginning of l^* (i.e., $beg = 1$) (Line 22). Then, we call Explore again to generate new trees from $roots$ in G_i (Lines 23-26). The mRR-set after cleaning is shown in Fig. 2d.

5 Related Work

Influence Maximization (IM). As a dual problem of AMCS, the IM problem is initiated in [8, 31] and formalized in [24], aiming at maximizing the influence spread under a budget constraint. Built on [3], TIM [37] and IMM [36] propose the reverse reachable sets (RR-sets) to estimate the influence of users, and solve IM with

a practical performance guarantee $1 - 1/e - \epsilon$. Then, Tang et al. devise the “select and verify” framework to reduce the number of RR-sets needed, and thus improve the efficiency [35]. Later, Guo et al. apply subset sampling to further boost IM algorithms [19]. Besides efficiency improvement, many works are also devoted to solving the IM problem from various perspectives, such as influence augmentation [6], neural networks [12], influence fairness [1, 11, 40], model-free IM [23], and competitive IM [5, 27, 38, 43].

MCSS. The MCSS problem is first studied in [28]. Inspired by [33], Goyal et al. propose a bi-criteria algorithm in [17], whose influence reaches $\eta - \alpha$ in expectation, where $\alpha > 0$ is an influence shortfall. Then, Zhu et al. solve the problem where the influence is not strictly submodular [42]. Han et al. propose the first practical algorithm that addresses the MCSS problem with performance guarantees [20]. Notably, Zhu et al. eliminate the influence shortfall [42]. However, the solution in [42] requires accurate influence estimation, which is #P-hard to compute. Later, Zhang et al. [41] and Feng et al. [9] address the influence shortfall issue, without relying on exact user influence. Moreover, [41] and [9] study a variant of MCSS, which strives to guarantee the probability that the influence reaches η is no less than a predefined value p , i.e., $\mathbb{P}[I(S) \geq \eta] \geq p$. However, we can never set $p = 1$ to address the AMCS problem, otherwise no estimation error can be tolerated.

ASMC. Different from the above works which belong to the non-adaptive category, Golovin and Krause take the head in studying the MCSS problem under the adaptive case [16]. Initially, they establish an approximation ratio for ASMC that is linear to $\ln \eta$. However, the ratio is found to contain loopholes [29], and revised to be quadratic to $\ln \eta$ [15]. Similar to [17, 20], Vaswani and Lakshmanan obtain an approximation ratio linear to $\ln \eta$ by allowing an influence shortfall [39]. Then, building on the theory in [15], Tang et al. develop the first practical algorithm called ASTI with performance guarantees [34]. However, the mRR-sets need to be regenerated from scratch in each round, incurring substantial computation overhead. On the other hand, the MAPLE algorithm is proposed to revise single-root RR-sets to solve the adaptive IM problem efficiently [18]. Nevertheless, MAPLE cannot solve AMCS with a guarantee.

6 Experiments

We evaluate the performance of our algorithm EMAS on a Linux server with an Intel Xeon 2.9 GHz CPU and 384 GB RAM. All algorithms are implemented in C++, compiled with g++ 9.4.0 under -O3 optimization, and executed single-threaded. Each algorithm is allotted up to three days to complete. The source code is in [10].

6.1 Experimental Setup

Datasets. We experiment on five real-world social networks: Epinions, DBLP, YouTube, LiveJournal, and Twitter, which are obtained from [25, 26]. Table 1 summarizes their statistics. For undirected networks DBLP and YouTube, each edge is treated as two directed edges to ensure the influence propagation is directional.

Algorithms. We evaluate six algorithms: the AMCS algorithm ASTI [34], the MCSS methods CLEAR [9] and SCORE [9], the adaptive IM approaches MAPLE [18] and ADAPTIM [21], and our proposed algorithm EMAS. ASTI is originally designed to solve the adaptive seed minimization problem where all user costs are 1.0.

Table 1: Datasets Information. ($K = 10^3, M = 10^6, B = 10^9$)

Dataset	n	m	Avg. Degree	Type
Epinions	131K	841K	6.39	directed
DBLP	317K	1.05M	6.62	undirected
YouTube	1.13M	2.99M	5.27	undirected
LiveJournal	4.85M	68.5M	14.1	directed
Twitter	41.7M	1.1B	27.3	directed

To adapt to AMCS, we allow ASTI to apply our idea of greedily selecting seeds based on the coverage-to-cost ratio. Thus, its cost is expected to be almost the same as ours. Following [9], we set the parameter ϵ_p of CLEAR as 0.01, which is an estimation error used to verify the attainment of η . For SCORE which guarantees $\mathbb{P}[I(S) \geq \eta] \geq p$, we set $p = 0.99$, and the error of probability estimation is $\epsilon_p = 1 - p = 0.01$. To adapt MAPLE and ADAPT-IM for AMCS, we revise the stopping criterion of their greedy policy from exhausting the budget to influencing at least η users. Moreover, for the single-node selection procedure of MAPLE, we continue selecting seed users until the expected influence exceeds η as in [18]. ADAPT-IM is designed for uniform user costs. Thus, we extend it to ADAPT-IM⁺, which selects seeds in the same way as EMASS.

Experimental Settings. We follow the settings widely adopted in prior studies. The diffusion probability for each edge is set as $p(u, v) = 1/|N_{\text{in}}(v)|$, where the expected number of influencers to v is 1 [24]. The estimation error is $\epsilon = 0.7$, enabling ASTI to produce results on all datasets. The failure probability is $\delta = 1/n_i$, ensuring a low probability of selecting unqualified seeds. Following [2, 7, 18], the user cost is set as $c(u) = 0.01 \times (1 + |N_{\text{out}}(u)|)$, where the additional 1 ensures nonzero cost. For adaptive algorithms, the number of seeds selected in each round is $b = 2$ for Epinions and DBLP, $b = 8$ for YouTube and LiveJournal, and $b = 4$ for Twitter. For non-adaptive algorithms, all seeds are selected in a single batch, so setting a batch size for them is unnecessary. The influence threshold η ranges from 2K to 10K for the three smaller datasets, from 10K to 50K for LiveJournal, and from 0.5M to 2.5M for Twitter.

We randomly generate 10 realizations for each dataset, and evaluate the adaptive algorithms in the fixed 10 realizations to report the averaged results. For the non-adaptive algorithms CLEAR, SCORE, and the non-adaptive procedure in MAPLE, we simulate 10,000 diffusion processes from the seed set selected by each algorithm to obtain the ratio of diffusions that exceed η . For the large datasets LiveJournal and Twitter, we can only perform 1,000 simulations to obtain the result in a reasonable time.

6.2 Experimental Results

Influence Attainment. We first verify whether the algorithms are feasible to AMCS. According to Def. 2.1, an algorithm is feasible only if η is reached in 100% simulations. Otherwise, the algorithm is infeasible and its other metrics are not meaningful. Fig. 3 reports the proportion of simulations where each algorithm attains η . Three algorithms (our EMASS, the modified ASTI and ADAPT-IM⁺) consistently achieve 100% attainment across all datasets and η values, since their seed selection will not stop until η users are observed to be influenced. Note that when $\eta > 30K$ in LiveJournal and $\eta > 1M$ in Twitter, ASTI produces no result in 3 days. Similar observation can be made on ADAPT-IM⁺. The non-adaptive algorithm SCORE attains η in most cases on Epinions and DBLP, but incurs high computation and user costs (see Figs. 4 and 5). Moreover, for YouTube,

LiveJournal, and Twitter in Fig. 3(c, d, e), SCORE runs out of memory (OOM) and fails to produce any results, since a large p leads to small error ϵ_p and tremendous graph samples for probability estimation. The MAPLE algorithm fails to attain η in most cases, as it frequently invokes a non-adaptive procedure. Even when MAPLE attains η (e.g., on YouTube), its seed cost is much higher than other algorithms. CLEAR does not attain η in most simulations and is thus infeasible for AMCS.

Seed Cost. Fig. 4 reports the total seed cost spent by each algorithm. The three feasible algorithms (i.e., our EMASS, ASTI, and ADAPT-IM⁺) achieve the lowest costs across all datasets. ASTI and ADAPT-IM⁺ attain nearly the same cost as EMASS, since they are adapted to select seeds in the same way as our EMASS. However, the runtime of ASTI and ADAPT-IM⁺ is much longer than EMASS, as shown in Fig. 5. MAPLE incurs the highest seed cost, as it is designed for IM rather than AMCS. Moreover, in Fig. 3, MAPLE fails to reach η in most simulations, making its high cost even less acceptable. CLEAR also incurs higher seed costs than EMASS and, more critically, fails to reach η in most simulations (see Fig. 3), limiting its practical utility. The high cost of CLEAR stems from its non-adaptivity. SCORE incurs significantly higher seed costs than EMASS on Epinions and DBLP, since it solves the MCSS problem non-adaptively, and selects many seeds based on pessimistic probability estimation. Similar to Fig. 3, SCORE is OOM on YouTube, LiveJournal, and Twitter.

Running Time. Note that CLEAR and MAPLE do not achieve 100% influence attainment in Fig. 3, which is a prerequisite for meaningful performance in AMCS. Therefore, their running times are not comparable, and are only reported in Fig. 5 for completeness. Compared with the SOTA algorithm ASTI, EMASS is much faster across all datasets. Specifically, EMASS achieves up to 41.2 $\times$ (YouTube) and 37.3 $\times$ (DBLP) speedup at $\eta = 10K$. Moreover, ASTI fails to terminate within the 3-day time limit on LiveJournal when $\eta > 30K$ (Fig. 5d) and on Twitter when $\eta > 1M$ (Fig. 5e). The superiority of EMASS over ADAPT-IM⁺ is even more significant, since ADAPT-IM⁺ is designed for IM which does not apply the mRR-sets tailored for AMCS. The high efficiency of EMASS is attributed to updating mRR-sets from previous rounds instead of regenerating them from scratch like ASTI and ADAPT-IM⁺. For SCORE, it runs OOM on YouTube, LiveJournal, and Twitter. In Epinions and DBLP, EMASS is also significantly faster than SCORE, with a maximum speedup up to 35.8 $\times$. Overall, Fig. 5 shows that EMASS is highly efficient for AMCS, while consistently attaining η with low seed costs (see Figs. 3 and 4).

The Linear Threshold (LT) Model. We further test the algorithms under the LT model [24] where a user becomes active if the aggregated weights from its active neighbors reach a threshold uniformly sampled from $[0, 1]$. The results are provided in Figures 6–7. Note that we omit the results of threshold attainment, since the performance of the algorithms is similar to the results under the IC model. The baseline ADAPT-IM is significantly inferior to ASTI and is thus not reported either. The results show that EMASS delivers similar improvement over existing methods, just like under the original IC model. CLEAR and MAPLE cannot attain the influence threshold η , with attainment rates far below 100% on most datasets, violating the constraint of AMCS and making their results unsuitable for comparison. Additionally, SCORE and MAPLE often incur

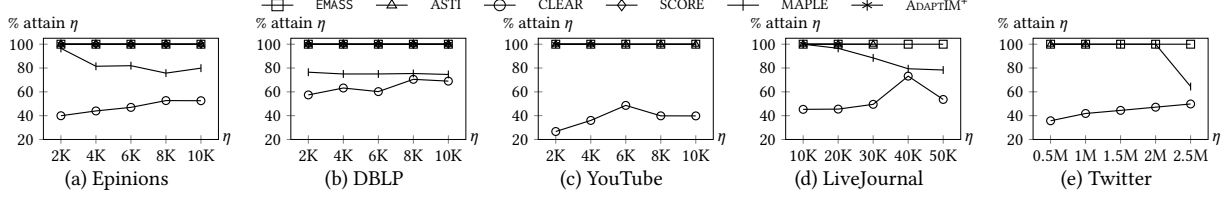


Figure 3: Percentage of influence attainment of each algorithm under the IC model.

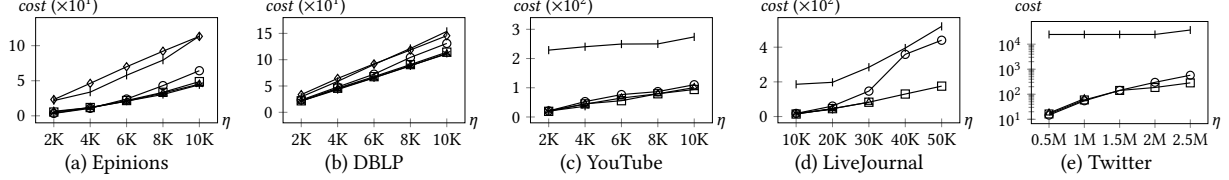


Figure 4: Seed cost of each algorithm under the IC model.

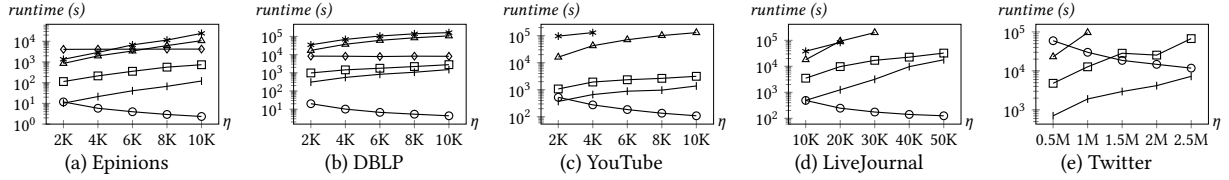


Figure 5: Running time of each algorithm under the IC model.

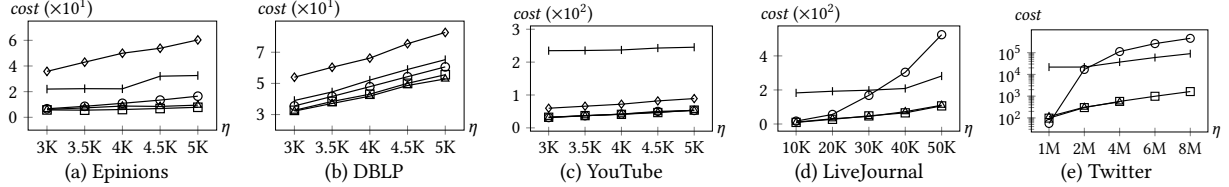


Figure 6: Seed cost of each algorithm under LT model.

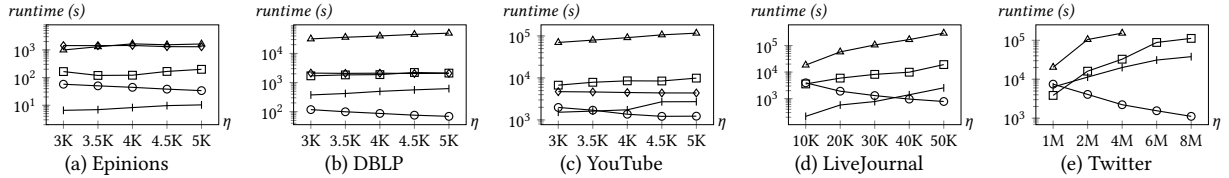


Figure 7: Running time of each algorithm under LT model.

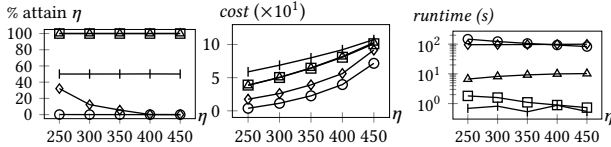


Figure 8: Results on the Congress network.

much higher seed costs than ASTI and EMASS. There is no result for SCORE on Livejournal and Twitter, since SCORE runs OOM. Compared to ASTI, EMASS is over 10 times faster in all datasets, e.g., achieving a 23.6× speedup at $\eta = 5K$ in DBLP. Moreover, ASTI cannot produce results on Twitter when $\eta > 4M$.

Congress Dataset. We further evaluate the algorithms under the Congress network [13], where there are 475 users and 13,289 edges, and the edge probabilities are obtained from historical interactions between members on Twitter. The results are presented in Fig. 8. As can be seen, CLEAR, SCORE, and MAPLE cannot always attain the influence threshold η , with percentages far below 100%. This is a prerequisite for meaningful performance in the AMCS problem definition, making their costs and runtimes not directly comparable.

Compared with ASTI, the only baseline that can always reach η , EMASS runs up to 14× faster while achieving nearly the same cost.

7 Conclusion

In this paper, we study the AMCS problem and propose a new framework called EMASS, which is efficient and offers performance guarantees. In EMASS, we greedily select seeds based on their coverage-to-cost ratio. To avoid regenerating mRR-sets, we develop a series of algorithms to update previous mRR-sets with correctness guarantees. Theoretical analysis shows that EMASS solves AMCS with an approximation ratio of $(\ln \eta + 1)^2 / [(1 - 1/e)^2(1 - \epsilon)]$. Extensive experiments demonstrate the effectiveness and efficiency of EMASS.

Acknowledgment

Jieming Shi is supported by the RGC grant (No. PolyU 15205224), funding from ByteDance, and SCRI P0051036-P0050643. Sibao Wang is supported by the RGC grant (No. 14217322), the “1+1+1” CUHK-CUHK(SZ)-GDST Joint Collaboration Fund (No. 2025A0505000045), and a CUHK Direct Grant. Chen Feng is supported by the postdoc matching fund scheme of PolyU (No. 1-W33U).

References

- [1] Florian Adriaens and Nikolaj Tatti. 2025. Fair Diversity Maximization with Few Representatives. In *KDD*. ACM, 17–25.
- [2] Song Bian, Qintian Guo, Sibao Wang, and Jeffrey Xu Yu. 2020. Efficient Algorithms for Budgeted Influence Maximization on Massive Social Networks. *Vldb* 13, 9 (2020), 1498–1510.
- [3] Christian Borgs, Michael Brautbar, Jennifer Chayes, and Brendan Lucier. 2014. Maximizing Social Influence in Nearly Optimal Time. In *SODA*. 946–957.
- [4] Damon Centola and Michael Macy. 2007. Complex contagions and the weakness of long ties. *Amer. J. Sociology* 113, 3 (2007), 702–734.
- [5] Guobang Chen, Wenjun Jiang, Kenli Li, Jingjing Wang, Jie Wu, and Kian-Lee Tan. 2025. Integrating Group Consensus for Competitive Influence Maximization in OSNs. *ACM Trans. Knowl. Discov. Data* 19, 9, Article 174 (Nov. 2025), 38 pages.
- [6] Xiaolong Chen and Jing Tang. 2025. Scalable Link Recommendation for Influence Maximization. In *KDD*. ACM, 130–141.
- [7] Artem Dogtiev. 2023. Influencer Marketing Costs. <https://www.businessofapps.com/marketplace/influencer-marketing/research/influencer-marketing-costs/>.
- [8] Pedro Domingos and Matt Richardson. 2001. Mining the network value of customers. In *KDD*. 57–66.
- [9] Chen Feng, Xingguang Chen, Qintian Guo, Fangyuan Zhang, and Sibao Wang. 2024. Efficient Approximation Algorithms for Minimum Cost Seed Selection with Probabilistic Coverage Guarantee. *Proc. ACM Manag. Data* 2, 4 (2024), 1–26.
- [10] Chen Feng, Gongyao Guo, Yiran Li, Jieming Shi, and Sibao Wang. 2026. Source Code adn Technical Report. <https://github.com/Planet-B612/AMCS>.
- [11] Yuting Feng, Ankitkumar Patel, Bogdan Cautis, and Hossein Vahabi. 2023. Influence Maximization with Fairness at Scale. In *KDD*. ACM, 4046–4055.
- [12] Yuting Feng, Vincent Y.F. Tan, and Bogdan Cautis. 2024. Influence Maximization via Graph Neural Bandits. In *KDD*. ACM, 771–781.
- [13] Christian G Fink, Kelly Fullin, Guillermo Gutierrez, Nathan Omodt, Sydney Zinnecker, Gina Sprint, and Sean McCulloch. 2023. A centrality measure for quantifying spread on weighted, directed networks. *Physica A* 626 (2023), 129083.
- [14] Malcolm Gladwell. 2006. *The Tipping Point: How Little Things Can Make a Big Difference*. Hachette UK.
- [15] Daniel Golovin and Andreas Krause. 2010. Adaptive Submodularity: Theory and Applications in Active Learning and Stochastic Optimization. arXiv:1003.3967 [cs.LG]
- [16] Daniel Golovin and Andreas Krause. 2011. Adaptive submodularity: Theory and applications in active learning and stochastic optimization. *J. Artif. Intell. Res.* 42 (2011), 427–486.
- [17] Amit Goyal, Francesco Bonchi, Laks VS Lakshmanan, and Suresh Venkatasubramanian. 2013. On minimizing budget and time in influence propagation over social networks. *Social Netw. Anal. Mining* 3 (2013), 179–192.
- [18] Qintian Guo, Chen Feng, Fangyuan Zhang, and Sibao Wang. 2023. Efficient Algorithm for Budgeted Adaptive Influence Maximization: An Incremental RR-set Update Approach. *Proc. ACM Manag. Data* 1, 3 (2023), 1–26.
- [19] Qintian Guo, Sibao Wang, Zhewei Wei, and Ming Chen. 2020. Influence maximization revisited: Efficient reverse reachable set generation with bound tightened. In *SIGMOD*. 2167–2181.
- [20] Kai Han, Yuntian He, Keke Huang, Xiaokui Xiao, Shaojie Tang, Jingxin Xu, and Liusheng Huang. 2019. Best bang for the buck: Cost-effective seed selection for online social networks. *IEEE Trans. Knowl. Data Eng.* 32, 12 (2019), 2297–2309.
- [21] Kai Han, Keke Huang, Xiaokui Xiao, Jing Tang, Aixin Sun, and Xueyan Tang. 2018. Efficient algorithms for adaptive influence maximization. *Vldb* 11, 9 (2018), 1029–1040.
- [22] Joe Hitchcock. 2025. Influencer Pricing: The Cost of Influencers in 2025. <https://www.shopify.com/blog/influencer-pricing>.
- [23] Alexandra Iacob, Bogdan Cautis, and Silviu Maniu. 2023. Sequential Learning Algorithms for Contextual Model-Free Influence Maximization. In *KDD*. ACM, 822–831.
- [24] David Kempe, Jon Kleinberg, and Éva Tardos. 2003. Maximizing the spread of influence through a social network. In *KDD*. 137–146.
- [25] Haewoon Kwak, Changhyun Lee, Hosung Park, and Sue Moon. 2010. What is Twitter, a social network or a news media?. In *WWW*. 591–600.
- [26] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data>.
- [27] Hui Li, Sourav S Bhowmick, Jiangtao Cui, Yunjun Gao, and Jianfeng Ma. 2015. Getreal: Towards realistic selection of influence maximization strategies in competitive networks. In *SIGMOD*. 1525–1537.
- [28] Cheng Long and Raymond Chi-Wing Wong. 2011. Minimizing seed set for viral marketing. In *ICDM*. 427–436.
- [29] Feng Nan and Venkatesh Saligrama. 2017. Comments on the Proof of Adaptive Stochastic Set Cover Based on Adaptive Submodularity and Its Implications for the Group Identification Problem in “Group-Based Active Query Selection for Rapid Diagnosis in Time-Critical Situations”. *IEEE Trans. Info. Theory* 63, 11 (2017), 7612–7614.
- [30] Pamela Oliver, Gerald Marwell, and Ruy Teixeira. 1985. A theory of the critical mass. I. Interdependence, group heterogeneity, and the production of collective action. *Amer. J. Sociology* 91, 3 (1985), 522–556.
- [31] Matthew Richardson and Pedro Domingos. 2002. Mining knowledge-sharing sites for viral marketing. In *KDD*. 61–70.
- [32] Everett M Rogers, Arvind Singhal, and Margaret M Quinlan. 2014. Diffusion of innovations. In *An integrated approach to communication theory and research*. Routledge, 432–448.
- [33] Petr Slavik. 1997. Improved performance of the greedy algorithm for partial cover. *Inf. Process. Lett.* 64, 5 (1997), 251–254.
- [34] Jing Tang, Keke Huang, Xiaokui Xiao, Laks VS Lakshmanan, Xueyan Tang, Aixin Sun, and Andrew Lim. 2019. Efficient approximation algorithms for adaptive seed minimization. In *SIGMOD*. 1096–1113.
- [35] Jing Tang, Xueyan Tang, Xiaokui Xiao, and Junsong Yuan. 2018. Online Processing Algorithms for Influence Maximization. In *SIGMOD*. 991–1005.
- [36] Youze Tang, Yanchen Shi, and Xiaokui Xiao. 2015. Influence maximization in near-linear time: A martingale approach. In *SIGMOD*. 1539–1554.
- [37] Youze Tang, Xiaokui Xiao, and Yanchen Shi. 2014. Influence maximization: near-optimal time complexity meets practical efficiency. In *SIGMOD*. 75–86.
- [38] Dimitris Tsaras, George Trimponias, Leftis Ntamos, and Dimitris Papadias. 2021. Collective influence maximization for multiple competing products with an awareness-to-influence model. *Vldb* 14, 7 (2021), 1124–1136.
- [39] Sharan Vaswani and Laks V. S. Lakshmanan. 2016. Adaptive Influence Maximization in Social Networks: Why Commit when You can Adapt? arXiv:1604.08171 [cs.SI]
- [40] Zhixiao Wang, Jiayu Zhao, Chengcheng Sun, Xiaobin Rui, and Philip S. Yu. 2024. A General Concave Fairness Framework for Influence Maximization Based on Poverty Reward. *ACM Trans. Knowl. Discov. Data* 19, 1, Article 14 (Dec. 2024), 23 pages.
- [41] Peng Zhang, Wei Chen, Xiaoming Sun, Yajun Wang, and Jialin Zhang. 2014. Minimizing seed set selection with probabilistic coverage guarantee in a social network. In *KDD*. 1306–1315.
- [42] Yuqing Zhu, Deying Li, and Zhao Zhang. 2016. Minimum cost seed set for competitive social influence. In *INFOCOM*. 1–9.
- [43] Jinhang Zuo, Xutong Liu, Carlee Joe-Wong, John CS Lui, and Wei Chen. 2022. Online competitive influence maximization. In *AISTATS*. PMLR, 11472–11502.

A Proofs

A.1 Notations

For readers’ convenience, here we summarize the frequently used notations in Table 2.

A.2 Proof of Theorem 3.1

To prove this theorem, we first establish the concentration inequality for mRR-sets. Given a collection of θ mRR-sets $\mathcal{R}$ and a set of seeds S , let $X_i = 1$ denote the event that the i -th mRR-set is covered by S , and $X_i = 0$ otherwise. Let $\bar{X} = \frac{1}{\theta} \sum_{i=1}^{\theta} X_i$. Then, for any $\epsilon \geq 0$, by [36] we have

$$\begin{aligned} \mathbb{P}[\bar{X} \geq (1 + \epsilon)\mathbb{E}[\bar{X}]] &\leq \exp\left(-\frac{\epsilon^2 \mathbb{E}[\bar{X}]\theta}{2 + 2\epsilon/3}\right), \\ \mathbb{P}[\bar{X} \leq (1 - \epsilon)\mathbb{E}[\bar{X}]] &\leq \exp\left(-\frac{\epsilon^2 \mathbb{E}[\bar{X}]\theta}{2}\right). \end{aligned} \quad (1)$$

Moreover, by [35], for any $a > 0$, we have

$$\begin{aligned} \mathbb{P}\left[\mathbb{E}[\bar{X}] \cdot \theta < \left(\sqrt{\bar{X} \cdot \theta + \frac{2a}{9}} - \sqrt{\frac{a}{2}}\right)^2 - \frac{a}{18}\right] &\leq e^{-a}, \\ \mathbb{P}\left[\mathbb{E}[\bar{X}] \cdot \theta > \left(\sqrt{\bar{X} \cdot \theta + \frac{a}{2}} + \sqrt{\frac{a}{2}}\right)^2\right] &\leq e^{-a}. \end{aligned} \quad (2)$$

Next, we distinguish two concepts like [34]: the candidate seeds, and the returned seeds. Since we adopt a “trial and error” manner for seed selection, the seeds selected in each trial may be unqualified and disposed, which are thus called candidate seeds. If the seeds selected at some trial are qualified and returned, they are the finally returned seeds. Note that EMAS stops at two conditions, we show

Table 2: Frequently used notations.

Notation	Description
$G = (V, E)$	the social network with users V and edges E
$G_i = (V_i, E_i)$	the residual graph used for the i -th round
n, m	the number of users and edges, respectively
$N_{\text{in}}(u), N_{\text{out}}(u)$	in-neighbors and out-neighbors of u , resp.
$p(e), p(u, v)$	diffusion probability of edge e or from u to v
η	the influence threshold
η_i	the remaining influence to be achieved in G_i
S	the set of seed users
S_i	the batch of seeds selected in round i
$\tilde{S}_i$	the total selected seeds after round i
$c(v), c(S)$	the cost of selecting a seed v or a seed set S
$I(S)$	a random influence spread of S
ϕ	a full realization
Φ	a random realization
$\Gamma_i(S)$	the truncated influence of S in G_i
$\tilde{\Gamma}(S)$	the estimated truncated influence of S
π	an adaptive seed selection policy
$\mathcal{R}$	a collection of mRR-sets
θ	the number of mRR-sets
k_i	the number of roots in an mRR-set at round i
$\Lambda(S)$	the number of mRR-sets covered by S
ϵ	the estimation error of $\mathcal{R}$ on $\Gamma(S)$
T_u	a BFS tree with root u
F	the array of all trees in R
L_u	the array of layer start positions in T_u
H	the array of all L for the trees in R

that under either condition, the returned seeds S are qualified, i.e.,

$$\mathbb{E}[\Gamma_i(S)] \geq (1 - 1/e)(1 - \hat{\epsilon})\mathbb{E}[\Gamma_i(S^o)]. \quad (3)$$

Case 1. $\theta > \theta_{\max}$.

Note that the first failure probability in Ineq. (1) is larger than the second one, under the same parameters. Thus, we only need to focus on the first one, which is stricter and implies the second one. Substituting $\theta_{\max}$ into Ineq. (1), we can see that $\mathbb{E}[\tilde{\Gamma}_i(S)] \geq (1 - \hat{\epsilon})(1 - 1/e)\mathbb{E}[\tilde{\Gamma}_i(S^o)]$ holds with probability at least $1 - \delta/3$. Further, by the estimation accuracy of mRR-sets, we derive

$$\begin{aligned} \mathbb{E}[\Gamma_i(S)] &\geq \mathbb{E}[\tilde{\Gamma}_i(S)] \geq (1 - \hat{\epsilon})(1 - 1/e)\mathbb{E}[\tilde{\Gamma}_i(S^o)] \\ &\geq (1 - \hat{\epsilon})(1 - 1/e)^2\mathbb{E}[\Gamma_i(S^o)]. \end{aligned}$$

Case 2. $\Lambda^l(S_i) \geq (1 - 1/e)(1 - \hat{\epsilon})\Lambda^u(S^o)$.

For the coverage of any seed set $S \subseteq V_i$, we construct the following two events:

$$E_1(S) : \mathbb{E}[\Lambda(S)] \geq \left(\sqrt{\Lambda(S) + \frac{2a_1}{9}} - \sqrt{\frac{a_1}{2}} \right)^2 - \frac{a_1}{18} = \Lambda^l(S),$$

$$E_2(S) : \mathbb{E}[\Lambda(S)] \leq \left(\sqrt{\Lambda(S) + \frac{a_2}{2}} + \sqrt{\frac{a_2}{2}} \right)^2 = \Lambda^u(S).$$

Note that the first event is about the lower bound calculated in EMAS, and the second event is about the upper bound. Applying Ineq. (2) and plugging the definitions of a_1 and a_2 , we see that

$$\mathbb{P}[\neg E_1(S)] \leq \delta/3W C_{n_i}^b,$$

$$\mathbb{P}[\neg E_2(S)] \leq \delta/3W.$$

Since the mRR-sets are randomly generated, the seeds returned also contain randomness. By the total probability, we have

$$\begin{aligned} \mathbb{P}[\neg E_1(S)] &= \sum_{S' \subseteq V_i} \mathbb{P}[\neg E_1(S')] \cdot \mathbb{P}[S' = S] \\ &\leq \frac{\delta}{3W} \sum_{S' \subseteq V_i} \mathbb{P}[S = S'] = \delta/3W. \end{aligned} \quad (4)$$

For the optimal seeds S^o , we derive

$$\mathbb{P}[\mathbb{E}[\Lambda(S^o)] > \Lambda^u(S^o)] = \mathbb{P}[\neg E_2(S^o)] \leq \delta/3W.$$

After replacing $\Lambda(S^o)$ in $E_2(S^o)$ with $\Lambda(S)/(1 - 1/e)$, the above inequality still holds, since $\Lambda(S)/(1 - 1/e) \geq \Lambda(S^o)$.

Summarizing the analysis for S and S^o , the following inequality holds with probability $1 - 2\delta/W$ for each seed selection trial.

$$\begin{aligned} \mathbb{E}[\Gamma_i(S)] &\geq \mathbb{E}[\tilde{\Gamma}_i(S)] = \mathbb{E}[\Lambda(S)]n_i/\theta \geq \Lambda^l(S)n_i/\theta \\ &\geq (1 - \hat{\epsilon})(1 - 1/e)\Lambda^u(S^o)n_i/\theta \geq (1 - \hat{\epsilon})(1 - 1/e)\mathbb{E}[\Lambda(S^o)]n_i/\theta \\ &= (1 - \hat{\epsilon})(1 - 1/e)\mathbb{E}[\tilde{\Gamma}_i(S^o)] \geq (1 - \hat{\epsilon})(1 - 1/e)^2\mathbb{E}[\Gamma_i(S^o)]. \end{aligned} \quad (5)$$

Taking all possible W trials into consideration, the seeds selected in each round satisfy the above inequality with probability at least $1 - 2\delta/3$.

Summing the failure probabilities in the two cases, we see that Ineq. 3 holds with probability $1 - \delta$. Let ω denote the randomness in selecting unqualified seeds. The expected influence achieved in each round i is

$$\begin{aligned} &\mathbb{E}_\omega[\mathbb{E}[\Gamma_i(S)]] \\ &\geq (1 - \hat{\epsilon})(1 - \delta)(1 - 1/e)^2\mathbb{E}[\Gamma_i(S^o)] \\ &= (1 - \epsilon)(1 - 1/e)^2\mathbb{E}[\Gamma_i(S^o)], \end{aligned}$$

where the last equality is due to the definition of $\hat{\epsilon}$.

With the approximation ratio achieved in each round of seed selection, we can derive the overall approximation ratio of EMAS by applying Theorem 40 in [15]

$$c(\text{EMAS}) \leq \frac{(\ln \eta + 1)^2}{(1 - 1/e)^2(1 - \epsilon)} c(\pi^o),$$

where π^o is the optimal policy.

A.3 Proof of Lemma 3.2

We first derive the time complexity of generating a single mRR-set, and then multiply it with the number of mRR-sets needed, to obtain the time complexity of the i -th round.

(1) To derive the time of a single mRR-set, we follow a similar idea to [34]. Let us consider an arbitrary node $v \in V_i$. Given a realization ϕ_i of the graph $G_i = (V_i, E_i)$, the influence of v is denoted as $x = I(v, \phi_i)$. Then, the probability of a random mRR-set not containing v is

$$p(\gamma) = \frac{C_{n_i - \gamma}^{k_i}}{C_n^k} = \prod_{h=0}^{k_i-1} \frac{n_i - \gamma - h}{n_i - h}. \quad (6)$$

For each node included into an mRR-set, its in-neighbors will be visited. Thus, the expected time of generating a random mRR-set is

$$\begin{aligned} \sum_{v \in V_i} d_{in}(v) \mathbb{E}_{k_i, \phi_i} [1 - p(\gamma)] &= \sum_{v \in V_i} d_{in}(v) \tilde{\Gamma}_i(v) / \eta_i \\ &= \sum_{v \in V_i} d_{in}(v) \frac{c(v)}{\eta_i} \frac{\mathbb{E}[\tilde{\Gamma}_i(v)]}{c(v)} \leq \sum_{v \in V_i} d_{in}(v) \frac{c(v)}{\eta_i} \frac{\mathbb{E}[\tilde{\Gamma}_i(v^\diamond)]}{c(v^\diamond)} \\ &\leq \frac{c_{\max} m_i}{\eta_i} \frac{\mathbb{E}[\tilde{\Gamma}_i(v^\diamond)]}{c(v^\diamond)} \leq \frac{c_{\max} m_i}{\eta_i} \frac{\mathbb{E}[\tilde{\Gamma}_i(v^\diamond)]}{c(v^\diamond)} \leq \frac{c_{\max} m_i}{\eta_i} \frac{\mathbb{E}[\tilde{\Gamma}_i(v^\diamond)]}{c(v^\diamond)}, \end{aligned}$$

where $v^\diamond$ is the node that maximizes the expected coverage-to-cost ratio, that is $v^\diamond = \arg \max_v \mathbb{E}[\tilde{\Gamma}_i(v)] / c(v)$, and the last inequality is due to the optimality of $v^\diamond$.

(2) To obtain the number of mRR-sets, we adopt the technique in [18], which specifies the number of RR-sets needed to reach the stopping condition of EMASS. Specifically, let $\theta_1 = \frac{2\eta_i \ln 2n_i}{\mathbb{E}[\tilde{\Gamma}_i(v^\diamond)] \epsilon_1^2}$. By a similar argument to the proof of Lemma A.4 in [18], when $\theta > \theta_1$, the inequality $\frac{\eta_i \Lambda(v^\diamond)}{\theta c(v^\diamond)} \geq (1 - \epsilon_1) \frac{\mathbb{E}[\tilde{\Gamma}_i(v^\diamond)]}{c(v^\diamond)}$ holds with probability $1 - 1/2n_i$. Further, due to the optimality of the selected node u^* , we have

$$\frac{\eta_i \Lambda(u^*)}{\theta c(u^*)} \geq (1 - \epsilon_1) \frac{\mathbb{E}[\tilde{\Gamma}_i(v^\diamond)]}{c(v^\diamond)}. \quad (7)$$

We say a node $v^\times$ is bad, if $\frac{\mathbb{E}[\tilde{\Gamma}_i(v^\times)]}{c(v^\times)} \leq (1 - \epsilon) \frac{\mathbb{E}[\tilde{\Gamma}_i(v^\diamond)]}{c(v^\diamond)}$. Let $\epsilon_2 = \epsilon - \epsilon_1$. By the concentration bound in Lemma ??, the probability that the estimated truncated influence of $v^\times$ deviates from its expectation by an error larger than $\epsilon_2 \frac{\mathbb{E}[\tilde{\Gamma}_i(v^\diamond)]}{c(v^\diamond)}$ is

$$\begin{aligned} &\mathbb{P} \left[\frac{\eta_i \Lambda(v^\times)}{\theta c(v^\times)} - \frac{\mathbb{E}[\tilde{\Gamma}_i(v^\times)]}{c(v^\times)} \geq \epsilon_2 \frac{\mathbb{E}[\tilde{\Gamma}_i(v^\diamond)]}{c(v^\diamond)} \right] \\ &\leq \mathbb{P} \left[\frac{\eta_i \Lambda(v^\times)}{\theta} - \mathbb{E}[\tilde{\Gamma}_i(v^\times)] \geq \epsilon_2 \frac{c_{\min}}{c(v^\diamond)} \mathbb{E}[\tilde{\Gamma}_i(v^\diamond)] \right] \\ &\leq \exp \left(- \frac{\epsilon_2^2 c_{\min} \mathbb{E}[\tilde{\Gamma}_i(v^\diamond)] / c(v^\diamond)}{2n_i (1 - \epsilon) + 2\epsilon_2 n_i / 3} \right) \\ &\leq \exp \left(- \frac{\epsilon_2^2 c_{\min} \mathbb{E}[\tilde{\Gamma}_i(v^\diamond)] / c(v^\diamond)}{2n_i} \right), \end{aligned} \quad (8)$$

where the second inequality is derived by applying the Chernoff bound and noting that $c_{\min} / c(v^\diamond) \leq 1$; the second inequality is due to $\epsilon_2 < \epsilon$ and the definition of bad nodes.

Let $\theta_2 = \frac{\eta_i (\ln 2 + 2 \ln n_i)}{\epsilon_2^2 c_{\min} \mathbb{E}[\tilde{\Gamma}_i(v^\diamond)] / c(v^\diamond)}$. When $\theta \geq \theta_2$, plugging θ to Ineq. (8), we see that the probability is no larger than $1/2n_i^2$. Since there are at most n bad nodes, by the union bound, the probability of a bad node is selected is at most $1/2n_i$. Accordingly, when $\theta \geq \max(\theta_1, \theta_2)$, with probability at least $1 - 1/n_i$, the selected node u^* is not a bad node, and satisfies

$$\begin{aligned} \frac{\mathbb{E}[\tilde{\Gamma}_i(u^*)]}{c(u^*)} &\geq \frac{n_i \Lambda(u^*)}{\theta c(u^*)} - \epsilon_2 \frac{\mathbb{E}[\tilde{\Gamma}_i(v^\diamond)]}{c(v^\diamond)} \\ &\geq (1 - \epsilon_1) \frac{\mathbb{E}[\tilde{\Gamma}_i(v^\diamond)]}{c(v^\diamond)} - \epsilon_2 \frac{\mathbb{E}[\tilde{\Gamma}_i(v^\diamond)]}{c(v^\diamond)} \\ &= (1 - \epsilon) \frac{\mathbb{E}[\tilde{\Gamma}_i(v^\diamond)]}{c(v^\diamond)}, \end{aligned} \quad (9)$$

where the second inequality is due to Ineq. (7), deduced based on $\theta \geq \theta_1$.

Further, by the estimation accuracy of mRR-sets, we have

$$\frac{\mathbb{E}[\Gamma(u^*)]}{c(u^*)} \geq (1 - 1/e)(1 - \epsilon) \frac{\mathbb{E}[\Gamma(v^\diamond)]}{c(v^\diamond)}.$$

To further analyze the number of mRR-sets needed, without loss of generality, we assume $\epsilon_2 \leq \sqrt{2}\epsilon_1$. Then, we can see that $\theta_2 \geq \theta_1$. When $\theta \geq \theta_2$, Ineq. (9) holds with probability $1 - 1/n_i$. For the failure cases, which happens with probability $1/n_i$, we need to double the mRR-sets. For the h -th doubling, we have $\theta = 2^h \theta_2$ and the probability that more doubling is needed is $1/n^{2^i}$. Then, the expected number of mRR-sets needed is

$$\mathbb{E}[\theta] \leq (1 - 1/n_i) \theta_2 + \sum_{h=1}^{\infty} \frac{2^h \theta_2}{n_i^{2^h}} = (1 - 1/n_i) \theta_2 + \frac{2}{n_i^2 - 2} \theta_2 \leq 2\theta_2.$$

Thus, the number of mRR-sets needed is at most $2\theta_2$.

Based on the above two parts, we can derive the expected time complexity by multiplying $2\theta_2$ and the time to generate a single mRR-set. Thus, the time complexity is $O\left(\frac{c_{\max}}{c_{\min}} \cdot \frac{(m_i + n_i) \ln n_i}{\epsilon^2}\right)$.

A.4 Proof of Theorem 3.3

To facilitate mRR-sets revision, the generation of mRR-sets is modified in Sec. 4.1, where we record the beginning position of each layer. To obtain the position, we need to check each node in the mRR-set. Thus, the time complexity of generating an mRR-set should further add the size of the mRR-set. However, it is often the case $m_i/n_i = \omega(1)$, thus the addition of n_i to $m_i + n_i$ is not significant and will not affect the complexity analysis.

Following the analysis in the proof of Lemma 3.2, the time complexity of generating an mRR-set is

$$\begin{aligned} &\sum_{v \in V_i} [d_{in}(v) + 1] \mathbb{E}_{k_i, \phi_i} [1 - p(\gamma)] \\ &= \sum_{v \in V_i} [d_{in}(v) + 1] \tilde{\Gamma}_i(v) / \eta_i \leq \frac{c_{\max} (m_i + n_i)}{\eta_i} \frac{\mathbb{E}[\tilde{\Gamma}_i(v^\diamond)]}{c(v^\diamond)}. \end{aligned} \quad (10)$$

Since EMASS only revises the mRR-sets, the number of mRR-sets needed will not change, and is still $\frac{\eta_i (\ln 2 + 2 \ln n_i)}{\epsilon_2^2 c_{\min} \mathbb{E}[\tilde{\Gamma}_i(v^\diamond)] / c(v^\diamond)}$. Accordingly, the time complexity of generating mRR-sets by our Alg. 2 is $O\left(\frac{c_{\max}}{c_{\min}} \cdot \frac{(m_i + n_i) \ln n_i}{\epsilon^2}\right)$.

Next, we show how many mRR-sets need to be revised. Note that, in EMASS the mRR-sets are inherited from the previous round. Often, the influence of the selected seed is small, and G_{i-1} and G_i are similar. Thus, for the convenience of analysis, we can assume the the numbers of mRR-sets needed in round $i - 1$ and round i are the same, and the numbers of roots needed in mRR-sets are also the same. Then, the time complexity of round i only depends on the revision process, which is determined by the number of mRR-sets we need to revise and the time to revise an mRR-set. Note that the time complexity of revising an mRR-set is upper bounded by the time of generating a new one, as shown in Ineq. (10).

Let the seed selected in the previous round be s_{i-1} , and the set of activated nodes in the previous round $i - 1$ be A_{i-1} . Given a realization ϕ_{i-1} based on the previous residual graph G_{i-1} , the number of nodes that can be influenced by s_{i-1} is $\gamma = I(s_{i-1}, \phi_{i-1}) = |A_{i-1}|$. For a random mRR-set R from the previous round, the probability that R is not polluted is equal to the probability that $R \cap A_{i-1} = \emptyset$.

It is also the probability that R does not contain any nodes in A_{i-1} , which can be expressed as $p(\gamma)$ as follows

$$\begin{aligned} p(\gamma) &= \frac{C_{n_{i-1}-\gamma}^{k_{i-1}}}{C_{n_{i-1}}^k} = \prod_{h=0}^{k_{i-1}-1} \frac{n_{i-1}-\gamma-h}{n_{i-1}-h} \\ &= \prod_{h=0}^{k_{i-1}-1} 1 - \frac{\gamma}{n_{i-1}-h} \geq 1 - \sum_{h=0}^{k_{i-1}-1} \frac{\gamma}{n_{i-1}-h}, \end{aligned} \quad (11)$$

where the inequality comes from the generalized Bernoulli's Inequality.

Then, the probability that an mRR-set needs to be revised is

$$1 - p(\gamma) \leq \sum_{h=0}^{k_{i-1}-1} \frac{\gamma}{n_{i-1}-h} \leq \frac{\gamma k_{i-1}}{n_{i-1}-k_{i-1}} = \frac{\gamma}{\eta_{i-1}-1}.$$

Thus, the expected number of mRR-sets to be revised is $\frac{\gamma}{\eta_{i-1}-1} \cdot \frac{\eta_i(\ln 2 + 2 \ln n_i)}{c_2^2 c_{\min} \mathbb{E}[\tilde{\Gamma}(v^\sigma)]/c(v^\sigma)}$. Multiplying the quantity with the time of generating an mRR-set Ineq. (10), we derive an upper bound of the time complexity of EMAS as follows

$$O\left(\frac{\gamma}{\eta_{i-1}-1} \cdot \frac{c_{\max}}{c_{\min}} \cdot \frac{(m_i + n_i) \ln n_i}{\epsilon^2}\right). \quad (12)$$

A.5 Proof of Lemma 4.1

Let the set of roots be K . In G_i , we generate two mRR-sets $R = \{F, H\}$ and $R' = \{F', H'\}$ from the roots K , following arbitrary different sequences of BFS probing. By the total probability, we can write the probability of $u \in F$ as

$$\mathbb{P}[u \in F] = \sum_{\phi} p_{\phi} \mathbb{P}[u \in F|\phi] = \sum_{\phi} p_{\phi} \mathbb{I}_{\{u \in F|\phi\}}.$$

Similarly, we can write $\mathbb{P}[u \in F'] = \sum_{\phi} p_{\phi} \mathbb{I}_{\{u \in F'|\phi\}}$. Note that, when the realization ϕ is fixed, all the states of edges are accordingly determined. Moreover, the values of the two indicator functions $\mathbb{I}_{\{u \in F|\phi\}}$ and $\mathbb{I}_{\{u \in F'|\phi\}}$ both depend on whether there is a live path from at least one root in K to the user u , which is determined by ϕ . As can be seen, the sequence of the probing of the edges does not affect the values of $\mathbb{I}_{\{u \in F|\phi\}}$ and $\mathbb{I}_{\{u \in F'|\phi\}}$. Thus, we have $\mathbb{I}_{\{u \in F|\phi\}} = \mathbb{I}_{\{u \in F'|\phi\}}$, and $\mathbb{P}[u \in F] = \mathbb{P}[u \in F']$.

A.6 Proof of Lemma 4.3

We consider a set of roots K , which contains k_i specific users of V_i . Let K_i be the set of k_i users randomly selected from G_i . Let K'_i be the k_i roots, k_{i-1} of which are randomly selected from G_{i-1} (denoted as K_{i-1}), and $\Delta k = k_i - k_{i-1}$ of which are randomly selected from G_i . Without loss of generality, we assume the k_{i-1} roots are not deleted from V_{i-1} . It is easy to see that the probability of K equal to K_i is $\mathbb{P}[K = K_i] = 1/C_{n_i}^{k_i}$. For the case of K'_i , if $K'_i = K$ holds, the k_{i-1} roots selected from V_{i-1} should also be in V_i . Otherwise, some roots will be deleted. Moreover, the k_{i-1} roots should be a subset of K , and the Δk roots selected from V_i should be $K \setminus K_{i-1}$ exactly. Then, we have $\mathbb{P}[K'_i = K] = C_{k_i}^{k_{i-1}} / [C_{n_i}^{k_{i-1}} C_{n_i-k_{i-1}}^{\Delta k}]$. We show that $\mathbb{P}[K_i = K] = \mathbb{P}[K'_i = K]$ as follows

$$\begin{aligned} \mathbb{P}[K_i = K] &= \mathbb{P}[K'_i = K] \\ \Leftrightarrow \frac{1}{C_{n_i}^{k_i}} &= \frac{C_{k_i}^{k_{i-1}}}{C_{n_i}^{k_{i-1}} C_{n_i-k_{i-1}}^{\Delta k}} \\ \Leftrightarrow \frac{k_i!(n_i - k_i)!}{n_i!} &= \frac{k_i!}{k_{i-1}!(\Delta k)!} \cdot \frac{k_{i-1}!(n_i - k_{i-1})!(\Delta k)!(n_i - k_{i-1} - \Delta k)!}{n_i!(n_i - k_{i-1})!} \\ \Leftrightarrow (n_i - k_i)! &= (n_i - k_{i-1} - \Delta k)!, \end{aligned}$$

where the last equality definitely holds due to the definition of Δk .

Thus, we prove that $\mathbb{P}[K_i = K] = \mathbb{P}[K'_i = K]$.

A.7 Proof of Lemma 4.4

Similar to the proof of Lemma 4.3, we assume the random mRR-set R' from G_{i-1} was not polluted by the diffusion from s_{i-1} . If R' was polluted, we will first resort to Alg. 4 to clean it, and the insertion of roots is directly done in G_i . Then, R' will definitely have the same distribution as a fresh mRR-set in G_i due to Lemma 4.1.

By Lemma 4.3, the roots of R' and R have the same distribution. Thus, we assume the roots of R' and R to be the same, and fix the roots to be K . Especially, the first k_{i-1} roots are denoted as K' , and the last $k_i - k_{i-1}$ roots are denoted as ΔK . W.l.o.g., we only consider the case where ΔK does not contain virtual roots. If ΔK contains virtual roots, we can start BFS from the virtual root to realize it. Observe that the realization of virtual roots will not bring new nodes into R , and that each node in the subtree from the virtual root will either be in the previous tree or the tree of the virtual root. That is, the realization of virtual roots only leads to the redistribution of previous nodes in the subtree from virtual roots. Under this assumption, to facilitate analysis, we stipulate a sequence for traversing roots, e.g., ascending order of their indices. In addition, we use F' to denote the forest corresponding to R' , which is generated from K' and ΔK sequentially. The sample space of F' is denoted as $\mathcal{F}'$, which contains all possible forests corresponding to R' . Similarly, the forest that corresponds to R is denoted as F , which is a sample from the collection $\mathcal{F}$. Meanwhile, the last $k_i - k_{i-1}$ trees of R (resp. R') are denoted as a forest ΔF (resp. $\Delta F'$). The first k_{i-1} trees of R (resp. R') are denoted as F^- (resp. F'^-).

Next, we would like to pinpoint that the sample spaces $\mathcal{F}' = \mathcal{F}$. To see this, we first recall that the roots of the trees in $\mathcal{F}$ and $\mathcal{F}'$ are the same. Note that, $\forall F'_s \in \mathcal{F}'$, the first k_{i-1} trees of F'_s are not polluted. Then, the reverse diffusions from K' will not activate any deleted users, and thus the users in F'_s will still be in G_i , which are also possible to be activated by reverse diffusions from H in G_i . Further, the diffusions from ΔH are directly conducted on G_i . Accordingly, the last $k_i - k_{i-1}$ trees of F' have the same possible results as F . Therefore, F and F' share the same sample space.

Based on the above arguments, we consider an arbitrary forest $F_s \in \mathcal{F}$. The first k_{i-1} trees of F_s are denoted as F_s^- , and the last $k_i - k_{i-1}$ trees are denoted as ΔF_s . Note that the generation of F_s (resp. F, F') from H also corresponds to some realizations, each of which is denoted as ϕ_s (resp. ϕ, ϕ'). Next, we derive the probability that the forest F' producing R' is exactly some forest $F_s \in \mathcal{F}$.

$$\begin{aligned} \mathbb{P}[F' = F_s] &= \sum_{\phi_s} p_{\phi_s} \prod_{e \in E_i} \mathbb{P}[\phi'(e) = \phi_s(e)] \prod_{e \in E_{i-1} \setminus E_i} \mathbb{P}[\phi'(e) = \text{observed state}] \\ &= \sum_{\phi_s} p_{\phi_s} \prod_{e \in E_i} \mathbb{P}[\phi'(e) = \phi_s(e)]. \end{aligned}$$

In the first equality, given the observation from the previous $i - 1$ rounds, we require the state of edges in $E_{i-1} \setminus E_i$ to be the same as the observation, since any other value is impossible. Due to the same reason, the probabilities are actually 1. Thus, we derive the second equation. For the fresh mRR-set R , the probability that its corresponding forest F is exactly F_s is $\mathbb{P}[F = F_s] = \sum_{\phi_s} p_{\phi_s} \prod_{e \in E_i} \mathbb{P}[\phi(e) = \phi_s(e)]$.

Note that the probabilities of $\phi(e)$ and $\phi'(e)$ being equal to some state are the same, only depending on the diffusion probability of e , i.e., $p(e)$. Thus, we have $\mathbb{P}[F' = F_s] = \mathbb{P}[F = F_s]$.

A.8 Proof of Lemma 4.6

Without loss of generality, we assume only one user w is deleted from the previous residual graph G_{i-1} , resulting in the current residual graph G_i . Next, we derive the probabilities of an arbitrary user $u \in V_i$ in R and R' . By the total probability, for a fresh random mRR-set $R = \{F, H\}$ from G_i , we have

$$\begin{aligned} \mathbb{P}[u \in F] &= \sum_{v \in V_i} \mathbb{P}[\text{user } v \text{ is selected as the root}] \mathbb{P}[u \in T_v] \\ &= \frac{1}{n_i} \sum_{v \in V_i} \mathbb{P}[u \in T_v] = \frac{1}{n_i} \sum_{v \in V_i} \sum_{\phi \in \Phi} p_{\phi} \mathbb{P}[u \in T_v | \phi], \end{aligned}$$

where T_v is the tree of root v . On the other hand, for the random mRR-set $R' = \{F', H'\}$ from G_{i-1} , R' should satisfy the premise that it was not deleted due to pollution of w . Then, R' can exist in round i to include the user u . Accordingly, by similar analysis to $\mathbb{P}[u \in F]$, we can derive the probability

$$\begin{aligned} &\mathbb{P}[u \in F'] \\ &= \mathbb{P}[u \in F', w \notin F'] \\ &= \sum_{v \in V_{i-1}} \mathbb{P}[\text{user } v \text{ is selected as the root}] \mathbb{P}[u \in T'_v, w \notin T'_v] \\ &= \frac{1}{n_{i-1}} \sum_{v \in V_{i-1}} \mathbb{P}[u \in T'_v, w \notin T'_v] = \frac{1}{n_{i-1}} \sum_{v \in V_{i-1}} \sum_{\phi \in \Phi} p_{\phi} \mathbb{P}[u \in T'_v, w \notin T'_v | \phi] \\ &= \frac{1}{n_{i-1}} \sum_{v \in V_i} \sum_{\phi \in \Phi} p_{\phi} \mathbb{P}[u \in T'_v, w \notin T'_v | \phi]. \end{aligned}$$

Note that, in the last equation of the above derivation, we change V_{i-1} to V_i . This equation still holds, since the probability of $w \in T'_v$ is 0 when w is selected as the root v . Further, given any realization ϕ , the probabilities $\mathbb{P}[u \in T_v | \phi]$ and $\mathbb{P}[u \in T'_v, w \notin T'_v | \phi]$ are actually either 1 or 0, since the states of edges under ϕ are deterministic. Moreover, given the same root v from V_i , $u \in T_v$ means that there is a path from v to u consisting of users in G_i . This path is also present in G_{i-1} , since V_{i-1} is a superset of V_i and the realization is the same. Conversely, $u \in T'_v$ while $w \notin T'_v$ means that there is a path from v to u , which does not contain w . Then, this path is also live in G_i . Thus, the two probabilities are equal to each other. Accordingly, we have $\mathbb{P}[u \in F] \neq \mathbb{P}[u \in F']$, while $\mathbb{P}[u \in F] = \frac{n_{i-1}}{n_i} \mathbb{P}[u \in F']$.

A.9 Proof of Lemma 4.7

The proof of this lemma is similar in spirit to that of Lemma 4.4. Here, we also assume no pollution occurs in R , since we are only interested in the distribution of roots when root removal is conducted. It is easy to see that the probability of selecting some specific $k - k'$ roots (denoted as K) from v_i is $1/C_{n_i}^{k-k'}$. For K' , K should be in K' , and k' other roots are selected from $V_{i-1} \setminus K'$. Accordingly, the probability for such case to happen is $C_k^{k'} / (C_{n_{i-1}}^k C_{n-k}^{k'})$. Note that $n_{i-1} = n_i$ here since no pollution is considered yet. Then, we only need to prove $1/C_{n_i}^{k-k'} = C_k^{k'} / (C_{n_{i-1}}^k C_{n-k}^{k'})$, which can be derived following a similar analysis in Sec. A.7.

A.10 Proof of Lemma 4.8

Without loss of generality, we assume the original mRR-set R_0 from G_{i-1} only contains one deleted user w , which is in the d -th layer of the k -th tree of R_0 . If w is a root in R_0 , by Alg. 4, we only need to delete the last $k_{i-1} - k$ trees, and insert $k_{i-1} - k$ new trees generated from G_i . This can be proved correct by Lemma 4.4.

Next, we consider the case where w is not a root of R_0 . Let $R_0^{\perp}$ be the result of truncating the part of R_0 after its d -th layer of the k -th tree, where w is located, and deleting w from this layer. Let $F'_{\perp}$ denote the forest composed of the trees of $R_0^{\perp}$, and $F_{\perp}$ denote the first k trees of R with the k -th tree of R truncated at the d -th layer. The corresponding mRR-set of $F_{\perp}$ is denoted as $R_{\perp}$. Note that R is an mRR-set directly generated from G_i .

$F'_{\perp}$ was not polluted by the diffusion in round $i - 1$. Hence, the proof of $F_{\perp}$ and $F'_{\perp}$ sharing the same distribution is similar to Sec. A.7, and we use the notations from Sec. A.7 if not redefined. By similar arguments in Sec. A.7, we see that $F'_{\perp}$ and $F_{\perp}$ have the same sample space denoted as $\mathcal{F}_{\perp}$, which includes all possible forests with roots from K in G_i . Then, the probability that $F'_{\perp}$ coincides with some forest $F_s \in \mathcal{F}_{\perp}$ is

$$\mathbb{P}[F'_{\perp} = F_s] = \sum_{\phi_s} p_{\phi_s} \prod_{e \in E_i} \mathbb{P}[\phi'(e) = \phi_s(e)].$$

On the other hand, the probability of $F_{\perp} = F_s$ is

$$\mathbb{P}[F_{\perp} = F_s] = \sum_{\phi_s} p_{\phi_s} \prod_{e \in E_i} \mathbb{P}[\phi(e) = \phi_s(e)].$$

Recall that the probabilities of $\phi'(e)$ and $\phi(e)$ being equal to some state are the same ($\forall e \in E_i$), as explained in Sec. A.7. We can conclude that $F'_{\perp}$ and $F_{\perp}$ have the same distribution. Accordingly, the distributions of $R_0^{\perp}$ and $R_{\perp}$ are also the same.

Fixing a sample from the distribution of $R_0^{\perp}$ and $R_{\perp}$, the d -th layers of the last trees of $R_0^{\perp}$ and $R_{\perp}$ are the same, denoted as l . Then, for $R_0^{\perp}$ and $R_{\perp}$, both sets of users in l start their reverse diffusions in G_i . The distribution will be definitely the same. By Alg. 4, we will generate new trees for $R'_{\perp}$ in G_i to derive the complete clean mRR-set R' . Due to the same root set, subsequent reverse diffusions from the rest $k_{i-1} - k$ roots in G_i also follow the same distribution as R .

Since each part of R' has the same distribution as R , and the cleaning process happens randomly, by the total probability, we can conclude that the cleaned mRR-set R' has the same distribution as an mRR-set R directly generated from G_i .